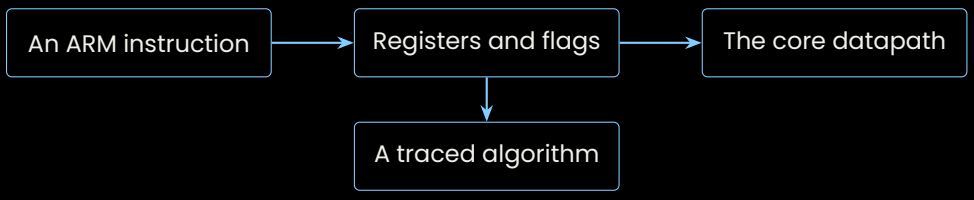
COLLEGE OF ENGINEERING TRIVANDRUM

Electrical and Computer Engineering / Semester 5

Microcontrollers and Embedded Systems

PCEOT501 / Module 4

ARM processors, architecture and assembly language programming



Architecture ARMv4T Core ARM7TDMI ARM state (A32)

Learn the architecture; design and trace the program

Complete syllabus coverage, vector diagrams, independently checked calculations, 17 complete assembly examples, answered exam practice, classroom corrections and a compact revision sheet. Both editions contain identical study material.

Pure black edition • Prepared 7 October 2026

Based on the official syllabus and Prof. Murali Krishnan's classroom material. Independently verified against Arm architecture manuals and assembler documentation. An independently prepared study aid, not an official CET publication.

Contents

Scope, assumptions and study method 4

1 ARM: architecture, core and microcontroller 5
1.1 The naming hierarchy . . . 5
1.2 ARM core and the TDMI suffix . . . 6
1.3 An ARM microcontroller in a system . . . 6
1.4 RISC versus CISC . . . 6
1.5 Advanced features in the classic ARM design . . . 7
1.6 Architecture versions and families . . . 8

2 Core architecture and instruction execution 9
2.1 The ARM7TDMI datapath . . . 9
2.2 Turn the execution animation into a trace . . . 9
2.3 The three-stage pipeline . . . 10
2.4 Instruction set architecture and encoding . . . 10

3 Registers, data and the CPSR 12
3.1 The sixteen visible registers . . . 12
3.2 Banked registers: visible names versus physical storage . . . 12
3.3 CPSR fields for ARMv4T . . . 13
3.4 Memory width, alignment and endianness . . . 13

4 Condition flags and conditional execution 14
4.1 What NZCV mean . . . 14
4.2 Worked flag laboratory . . . 15
4.3 The complete condition-code reference . . . 15
4.4 Conditional instructions and stale flags . . . 16
4.5 Logical, shift and multiply flags . . . 16

5 Operating modes, exceptions and mode switching 17
5.1 Why there are seven modes . . . 17
5.2 The ARM7TDMI exception vectors . . . 17
5.3 Exception entry: what hardware saves . . . 18
5.4 Software mode switching and initialized stacks . . . 19
5.5 IRQ handler structure and nesting . . . 19

6 Writing and tracing real ARM assembly 20
6.1 Read an instruction before memorizing it . . . 20
6.2 Source directives and the complete-program wrapper . . . 21
6.3 How to assemble and inspect an example . . . 21
6.4 Immediate constants and literal loads . . . 22
6.5 Shifts, rotates and the barrel shifter . . . 22
6.6 Branches, calls and a stack . . . 23
6.7 Loads/stores and indexed addressing . . . 23

7 Arithmetic ALPs: design, code and results 24
7.1 Addition and the register plan . . . 24
7.2 Subtraction, reverse subtraction and borrow . . . 25
7.3 Two-word addition and subtraction . . . 26

7.4 Multiplication and its ARM7 restrictions . . . 27
7.5 A full 64-bit product and constant multiplication . . . 28

8 Logical ALPs and bit manipulation 29
8.1 Bitwise operations and masks . . . 29
8.2 Set, clear, toggle and extract . . . 29
8.3 Memory-mapped peripheral application . . . 30
8.4 Count set bits through the shifter carry . . . 30
8.5 Reverse four bits and find a GCD . . . 31

9 Division without a hardware divider 33
9.1 Specify the problem and its boundary cases . . . 33
9.2 Repeated subtraction: the clearest first ALP . . . 33
9.3 Binary shift-and-subtract, first in eight bits . . . 34
9.4 A complete 32-bit routine with the transient carry . . . 35
9.5 Power-of-two division and signed rounding . . . 36

10 Factorial: loops, bounds and a subroutine 36
10.1 Translate the algorithm into a register plan . . . 36
10.2 Complete bounded factorial program . . . 37
10.3 Why 12 is the largest safe one-word factorial . . . 38
10.4 A callable factorial with a saved stack frame . . . 38

11 Answered exam and classroom practice 40
11.1 Forty short questions with model answers . . . 40
11.2 Ten developed 9-mark practice answers . . . 44
11.3 Six classroom laboratories: complete record mapping . . . 46
11.4 Six debugging exercises with corrected diagnoses . . . 47
11.5 All fifteen graded practice problems, completed . . . 48
11.6 Classroom quiz and demonstrations: where each answer lives . . . 49

12 Final revision sheet and completion checklist 50
12.1 Syllabus completion and self-test checklist . . . 51

13 Discrepancies and deliberate teaching simplifications 52

14 Chapter-mapped sources and further reading 55
14.1 Official scope and local teaching sources . . . 55
14.2 Architecture, device and tool references . . . 55
14.3 Prescribed textbooks: honest access and chapter mapping . . . 56
14.4 What to open for each chapter . . . 57

Scope, assumptions and study method

The official syllabus controls coverage. PCEOT501 starts on physical PDF page 128; the Module 4 row is on page 129 and has **12 contact hours**. It covers introduction to ARM, core and microcontroller, RISC versus CISC, advanced features, architecture versions, architecture and ISA, operating modes, register set and general-purpose registers, mode switching, conditional flags, and simple arithmetic/logical ALPs including addition, subtraction, multiplication, division and factorial. The 8051 interfacing and motor topics belong to Module 3.

Exam pattern. The ESE is **60 marks in 2 hours 30 minutes**. Part A: all eight questions, two per module, 3 marks each. Part B: two full questions per module; choose one, worth 9 marks, with at most three subdivisions. This module therefore supplies two 3-mark answers and one chosen 9-mark answer. Practice here follows that format; it is not a prediction or an official marking scheme.

Processor and assembler contract

The default is **ARM7TDMI implementing ARMv4T**, **32-bit ARM state**, 32-bit registers and **little-endian, aligned memory**. "A32" is convenient modern terminology for the classic ARM instruction set. Original Thumb is introduced for comparison; Cortex-M and AArch64 details are clearly separated.

Complete files use **GNU assembler syntax**, explicitly selecting `.cpu arm7tdmi` and `.arm`. They are independent bare-metal examples, linked at a test address and entered by a debugger/emulator. They are not board startup images. Each finishes at `done: B done`, where registers can be inspected. `HALT` is a classroom simulator command, not an ARM7 instruction. Keil `armasm` directives are compared separately; 8051 C51 is not an ARM assembler.

Study order. Learn the naming hierarchy and RISC trade-offs; trace the core and pipeline; understand registers, flags and signed/unsigned comparisons; then study modes and exceptions. Learn assembly syntax and operand restrictions before the algorithms. For each program, write the register plan, initial values, loop invariant, termination test and expected output. Attempt the questions before reading their answers.

Verification boundary. All 17 complete programs and the labelled IRQ fragment were assembled for ARM7TDMI. Machine-code examples were executed in Unicorn's TI925T ARMv4T model against independent Python integer oracles. This checks functional examples, not physical ARM7TDMI timing, electrical operation or a vendor's interrupt controller. No board or licensed Keil ARM toolchain was available. See `VALIDATION.md` for exact completed checks.

Syllabus-to-page map

Printed page numbers count the cover as page 1 and match physical PDF pages.

Syllabus item	Section	Page
Introduction to ARM processors	1	5
ARM core	1.2	6
ARM microcontroller	1.3	6
RISC versus CISC	1.4	6
Advanced features of ARM	1.5	7
Architecture versions	1.6	8
ARM architecture	2	9

Syllabus item	Section	Page
Instruction set architecture	2.4	10
Operating modes	5	17
Register set	3	12
General-purpose registers	3.1	12
Mode switching	5.3	18
Conditional flags	4	14
Arithmetic ALPs; addition	7, 7.1	24
Subtraction	7.2	25
Multiplication	7.4	27
Logical operations	8	29
Division	9	33
Factorial	10	36

Classroom continuity. The entire 23-slide HTML and JavaScript were inspected: six labs, six debugging exercises, 15 graded practice problems, 28 quiz questions, the instruction engine and interactive architecture/register/flag/mode/algorithm demonstrations. These become static diagrams, traces, answers and corrected examples. The existing 19-page `Module 4/Module-4.pdf` was also reviewed as a supplementary local summary; primary references resolve conflicts. Every lab and graded practice item is mapped in the practice chapter. Public textbook contents were consulted where available; unavailable chapter bodies are not claimed as read.

1 ARM: architecture, core and microcontroller

1.1 The naming hierarchy

An **architecture** specifies what software can observe: instructions, registers, data formats, privilege and exceptions. A **core** is hardware implementing that specification. An **ARM microcontroller** integrates a core, memory and peripherals into a chip that can control an embedded product. The product adds power, a board, sensors, drivers and firmware. "Processor" is a broader product term; vendors may use it for the core or a subsystem containing a core and related blocks.

Layer	Example	What the name tells you
Architecture	ARMv4T	Programmer-visible rules; "v" indicates a version.
Core	ARM7TDMI	A particular implementation of ARMv4T.
Processor subsystem	ARM720T	ARM7-family core with cache/MMU integration.
Microcontroller	NXP LPC2148	ARM7TDMI-S plus Flash, SRAM, buses and peripherals.
Embedded system	Motor controller	MCU, driver, motor, feedback and control software.

Arm supplies processor architecture and IP designs; silicon vendors integrate or implement them in chips. Some licensees design their own compatible cores. Instruction compatibility does not

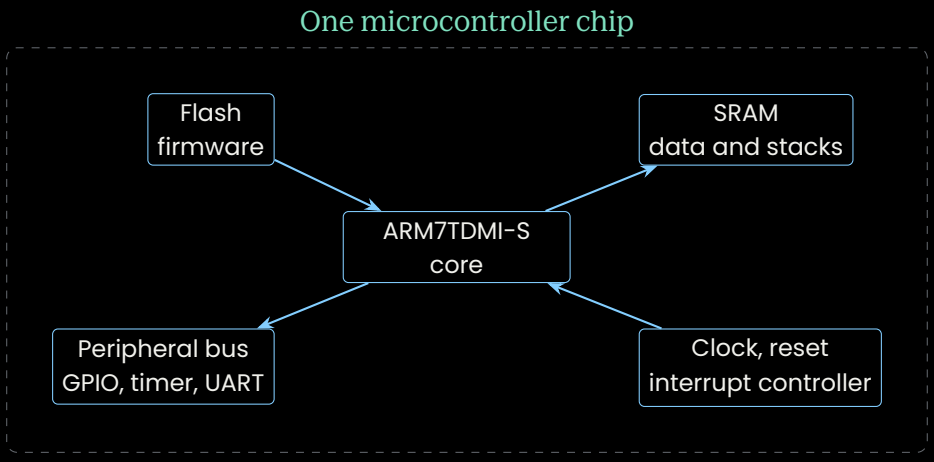
make peripheral addresses, clock registers or pin functions interchangeable. A Cortex-M3 device and an ARM7TDMI device also differ in instruction state and exception model.

1.2 ARM core and the TDMI suffix

The core fetches and decodes instructions, reads registers, computes results and issues memory transfers. In **ARM7TDMI**: **T** is Thumb support, **D** is on-chip debug support, **M** is an enhanced multiplier and **I** is EmbeddedICE support. ARM7 is the core family name; it does not mean ARMv7. ARM7TDMI-S is a synthesizable implementation used in microcontrollers such as the LPC2148.

1.3 An ARM microcontroller in a system

The LPC2148 is a concrete example: ARM7TDMI-S core, 512 KB Flash and 32 KB main SRAM plus 8 KB USB-accessible SRAM, peripheral buses, GPIO, timers, UARTs, USB and ADC/DAC functions. These are **device features**, not instructions built into ARMv4T. The datasheet specifies power, pins, memory and peripherals; the architecture manual specifies instructions and register behaviour.



An MCU writes a logic-level motor command through GPIO; a driver powers the motor. An ADC places a conversion result in a memory-mapped register; an `LDR` reads it. A timer signals an interrupt; the interrupt controller presents IRQ/FIQ to the core. The core does not determine motor current, ADC scaling or peripheral acknowledgement rules.

Verification and further reading: [TRM] introduction; [NXP] features and block diagram; [FURBER] chapters 2, 9 and 13.

1.4 RISC versus CISC

RISC means Reduced Instruction Set Computer; **CISC** means Complex Instruction Set Computer. The useful contrast is how operations are exposed to software and implemented, rather than a claim that one style always wins. Classic ARM emphasizes regular instruction encoding, register computation and explicit memory transfers.

Feature	Classic ARM (RISC example)	x86 (CISC example)
Instruction length	ARM state: 32 bits; original Thumb mostly 16 bits.	Variable-length encodings.
ALU operands	Registers/immediate; load data first.	Many operations permit a memory operand.
Register model	16 visible registers, with special PC/SP/LR roles.	Different register counts/roles across x86 generations.
Decode	Regular fields support straightforward decode.	Rich encodings require more decode work.
Program size	May need several simple instructions; Thumb improves density.	A complex instruction can express several steps.
Implementation	ARM7 is a small in-order pipelined core.	Modern cores may translate instructions into internal operations.
Timing and power	Depend on instruction, memory and implementation.	Also depend on microarchitecture and workload.

Load, operate, store

To add `r0` to a RAM word addressed by `r4`, ARM uses:

```
LDR r1, [r4]       @ memory -> register
ADD r1, r1, r0     @ compute using register operands
STR r1, [r4]       @ register -> memory
```

`ADD r0,r1,r2` cannot read `r2` directly from memory. This does not mean that only two mnemonics ever access data memory: byte/halfword transfers, `LDM/STM` and `SWP` are also present. Instruction fetch itself reads memory.

A CISC-style `ADD [total],eax` expresses a read, addition and write in one instruction. Count of source instructions alone does not prove speed, energy use or byte count. The classroom `REP LODSD` array example loads repeatedly but **does not accumulate an array sum**; the ARM sum loop is developed in Section 6.7.

1.5 Advanced features in the classic ARM design

1. **Conditional execution:** nearly all ARMv4T ARM-state instructions carry a condition field and can execute only if CPSR flags match.
2. **Barrel shifter:** operand 2 can be shifted/rotated before the ALU, e.g. `ADD r0,r1,r2,LSL #2` computes $r_1 + 4r_2$ in one instruction.
3. **Pipeline:** fetch, decode and execute overlap to improve throughput.
4. **Thumb:** a compact instruction set reduces code size and can help systems with narrow memory buses; switching uses `BX`.
5. **Banked registers and exception state:** handlers get private SP/LR and an SPSR; FIQ has extra private working registers.

6. **Block transfers and addressing:** `LDM/STM` move register groups; indexed addressing can update a pointer as part of a transfer.
7. **Multiply and debug:** hardware multiplication and EmbeddedICE support computation and development.

Do not add a cache, MMU, FPU, NVIC or hardware divider to the bare ARM7TDMI. Some other cores/MCUs provide additional blocks. An immediate shift on a normal ARM7 ALU operation normally adds no separate instruction or extra internal shift cycle; a **register-controlled shift adds an internal cycle**. "Every shift is free" is too broad.

1.6 Architecture versions and families

The table gives relationships to learn, without treating a core's launch year as an architecture's defining feature.

Architecture	Representative core	Distinction
ARMv4T	ARM7TDMI	ARM and original Thumb; seven modes; no hardware divide.
ARMv5TE / v5TEJ	ARM926EJ-S (v5TEJ)	Enhanced DSP operations; **J** adds Jazelle. v5TE alone does not imply Jazelle.
ARMv6	ARM1136J-S	Media/SIMD and other extensions. Thumb-2 belongs to **v6T2**, not every v6 core.
ARMv6-M	Cortex-M0/M0+	Small M-profile, Thumb instruction subset; no A32 or hardware divide.
ARMv7-M	Cortex-M3	Thumb/Thumb-2; Thread/Handler modes; NVIC; hardware divide.
ARMv7E-M	Cortex-M4/M7	M-profile with added DSP instructions; not the classic seven-mode model.
ARMv7-A/R	Cortex-A8 / Cortex-R4	Application/real-time profiles; A32/T32 capabilities and feature options depend on implementation.
ARMv8-A	Cortex-A53	Adds AArch64/A64 and its different 64-bit register model. AArch32 availability depends on implementation.
ARMv8-M	Cortex-M23/M33	M-profile security extensions; features differ between Baseline/Mainline cores.
ARMv9-A	Modern A-profile cores	Later application architecture; included only for orientation.

Profiles: A targets application systems, commonly with an MMU and operating system; R targets real-time processing; M targets microcontrollers. These are related families, not a guarantee of identical state, vectors, instructions or timing. Original ARMv4T Thumb has conditional branches but no `IT` instruction; Thumb-2-era conditional execution must not be projected backward.

3 marks: distinguish architecture, core and MCU.

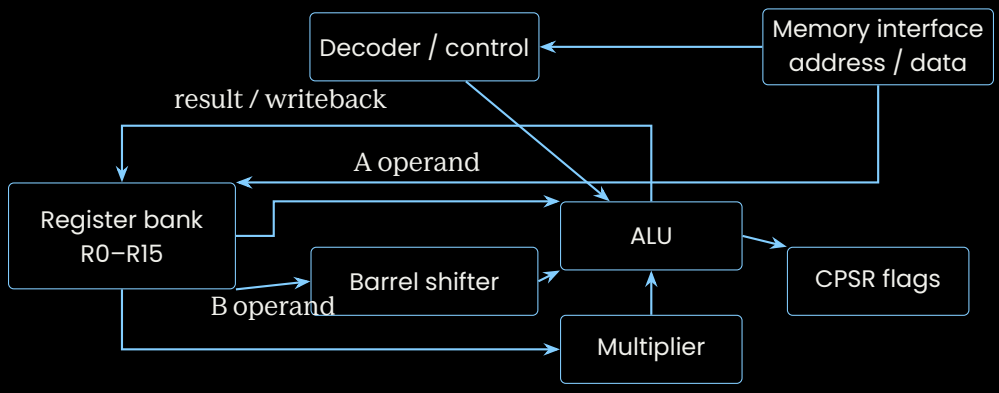
Model answer. ARMv4T is an architectural specification; ARM7TDMI is a core implementing it; LPC2148 is a complete MCU integrating ARM7TDMI-S with memory and peripherals. The first fixes software rules, the second implements the CPU, and the third fixes the device's memory/peripheral interfaces.

Verification and further reading: [TRM] introduction; [ARCH] architecture-version notes; [PROFILE] Arm core/architecture mapping; [FURBER] chapters 1–2 and 9.

2 Core architecture and instruction execution

2.1 The ARM7TDMI datapath

The core has a register bank, ALU, barrel shifter, multiplier, instruction decoder/control, address generation and memory interface. ARM7TDMI uses a **unified instruction/data memory interface** (a von Neumann-style organization). This is a property of this core; other ARM cores can use separate instruction/data paths. Load/store describes how *instructions access data*, not whether buses are Harvard or unified.



Block	Purpose in execution
Register bank	Supplies operands; accepts a destination result; banking changes selected physical registers.
Barrel shifter	Forms operand 2 by shifting/rotating; provides shifter carry for logical flag-setting instructions.
ALU	Arithmetic/logic result and appropriate condition flags.
Multiplier	Low-word or long multiply operations; timing can depend on multiplier value.
Decoder/control	Decodes fields, checks the condition, controls register writes and memory cycles.
Address generation	Forms fetch/data addresses and pointer updates.
Memory interface	Transfers instructions and data; bus wait states can extend execution.

2.2 Turn the execution animation into a trace

For `ADD r0,r1,r2`, let $r_1 = 7, r_2 = 3$. Fetch obtains the encoded instruction. Decode identifies ADD, destination R0 and two source registers. Register reads supply 7 and 3; an unshifted operand 2 passes unchanged. The ALU computes 10; writeback puts 10 in R0. R1/R2 remain unchanged because they are distinct source registers. Without S, NZCV remain as before.

Worked example: The source and destination can overlap

`ADD r1,r1,r2` reads the old R1/R2 values and then writes R1. Starting with 7 and 3, the final R1 is 10 and R2 is 3. "Source registers never change" is true only when they are not also destinations (and for loads, not a writeback base).

`CMP r1,r2` similarly computes a subtraction, but discards the numerical result and writes flags. A failed instruction condition suppresses its architectural effects. The hardware stage narrative is not a software routine that must be written for every ADD.

2.3 The three-stage pipeline

Fetch reads the instruction; **decode** identifies it and selects operands; **execute** completes its operation. Overlap improves throughput.

Instruction	Cycle 1	2	3	4	5	6
I_1 at 0x8000	F	D	E			
I_2 at 0x8004		F	D	E		
I_3 at 0x8008			F	D	E	
I_4 at 0x800C				F	D	E

In an **ideal sequence of single-cycle execute operations with no stalls or branches**, n instructions require $n + 2$ stage cycles; ten require twelve. Latency and throughput differ: a simple instruction crosses three stages, yet after filling, one can finish per cycle. Loads, stores, multiply, register-controlled shifts, branches and slow memory do not all fit that ideal model.

Worked example: PC value and a branch refill

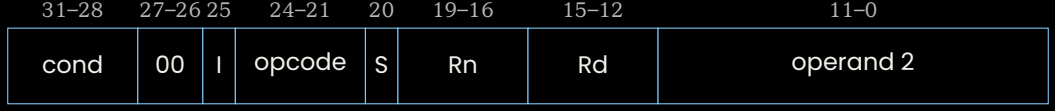
While the ARM instruction at 0x8000 executes, subsequent instructions are in decode/fetch. A normal read of PC gives **0x8008**. A taken branch replaces the sequential stream and refills the pipeline. A32 `B/BL` use the architectural PC base (instruction address $+8$), not simply the address of the branch itself.

PC reads $A + 8$ for ordinary ARM-state operand/address uses and typically $A + 4$ in Thumb. ARM7 has special cases, such as PC in a register-controlled shift operand or a stored PC value; avoid these in beginner ALPs. The $+8$ rule is an architectural convention retained by many deeper-pipeline A32 cores, not a formula for counting their pipeline stages.

2.4 Instruction set architecture and encoding

An ISA defines legal instructions, their effects, registers, operand/addressing rules, data sizes and exceptions. Microarchitecture describes the hardware implementing those rules. Compatibility requires the same supported instruction subset, execution state, environment and ABI; A32 code is not directly executable on a Thumb-only Cortex-M or in AArch64.

For an ordinary data-processing instruction, the ARM-state 32-bit word has this format. Multiplication, branch and transfer instructions use different encodings.



`cond=1110` is AL (always); $I = 1$ selects an immediate; opcode identifies the operation; S controls flags; Rn is the first source; Rd is the destination. With $I = 0$, operand 2 describes a register and optional shift. With $I = 1$, it is an 8-bit constant rotated right by an even amount.

Worked example: Encode the classroom instructions

Instruction	Essential fields	Machine word
`ADD r0,r1,r2`	AL, ADD opcode 0100, I=0, S=0, Rn=1, Rd=0, Rm=2	`E0810002`
`MOV r3,#10`	AL, MOV opcode 1101, I=1, S=0, Rd=3, rotation=0, imm=10	`E3A0300A`
`SUBS r4,r5,r6`	AL, SUB opcode 0010, I=0, S=1, Rn=5, Rd=4, Rm=6	`E0554006`

For example ADD's word is $(14 \ll 28)+(4 \ll 21)+(1 \ll 16)+2$. Encodings were independently calculated and compared with assembled bytes. Memory byte order is separate: `E0810002` is stored as `02 00 81 E0` in little-endian memory.

ARMv4T uses condition values 0000–1110; 1111 is reserved and should not be used as an ordinary condition. Later A32 versions reuse some encodings in an unconditional space, so "every ARM instruction is conditional" requires an architecture qualifier.

3 marks: explain pipelining and the PC offset.

Model answer. ARM7TDMI overlaps fetch, decode and execute. In an ideal filled pipeline, one simple instruction completes per cycle. During execution the fetch position is two 4-byte ARM instructions ahead, so an ordinary PC read is the current instruction address plus 8. Branches and memory stalls disturb ideal throughput.

Verification and further reading: [TRM] sections 1.1–1.4 and instruction-cycle chapter 6; [ARCH] ARM instruction encodings and data-processing addressing; [FURBER] chapters 2, 4 and 5.

3 Registers, data and the CPSR

3.1 The sixteen visible registers

Every visible core register is 32 bits. R0–R12 are general-purpose registers; R13, R14 and R15 have conventional/architectural special roles. This is **16 registers total**, not sixteen general-purpose registers plus another three.

Register(s)	Name	Role
R0–R3	Arguments / scratch	Common ABI argument/result registers; caller saves values needed across a call.
R4–R11	General registers	Mainly preserved variables under the ABI; R9's role is platform-dependent.
R12	IP / scratch	Intra-procedure scratch; may be used by linker-generated veneers.
R13	SP	Stack pointer; banked in exception modes.
R14	LR	Link register; BL writes a return address; banked in exception modes.
R15	PC	Program counter; read/write behaviour differs from ordinary data registers.
CPSR	Current status	Flags, interrupt masks, instruction state and mode.
SPSR (when present)	Saved status	Exception mode's saved CPSR for return.

Argument conventions are **software agreements**, not wiring requirements. A register has no inherent signedness: `FFFFFFFF` can mean unsigned 4,294,967,295 or signed -1. The instruction/condition and the algorithm determine interpretation. Register edits in the classroom explorer simply alter an input value; they do not configure a peripheral.

3.2 Banked registers: visible names versus physical storage

Banking selects a private physical copy when mode changes. Shared names keep their current contents. User and System share a complete register bank. FIQ has private R8–R14; IRQ, Supervisor, Abort and Undefined each have private R13/R14.

Group	User	System	FIQ	IRQ	SVC	Abort	Und.
R0–R7	Shared	Shared	Shared	Shared	Shared	Shared	Shared
R8–R12	Shared	Shared	Private	Shared	Shared	Shared	Shared
R13/R14	User	User	FIQ	IRQ	SVC	Abort	Und.
R15	Shared	Shared	Shared	Shared	Shared	Shared	Shared
CPSR	One current status register						
SPSR	None	None	FIQ	IRQ	SVC	Abort	Und.

Worked example: Why the total is 37

Sixteen base core registers, seven extra FIQ copies, eight extra SP/LR copies for four other exception modes, one CPSR and five SPSRs give

$$16 + 7 + 4(2) + 1 + 5 = 37.$$

That is **31 physical core registers plus 6 status registers**. Only 16 core registers are visible at once. An exception mode additionally exposes its SPSR; User/System have only CPSR.

Switching from User to IRQ changes which SP/LR are visible but does not copy the User stack into the IRQ stack. Startup must initialize each stack before use. Nested interrupts into the *same* mode reuse that mode's LR/SPSR and can overwrite earlier saved state unless software protects it.

3.3 CPSR fields for ARMv4T



- **N/Z/C/V** describe flag-setting results (next chapter).
- **I=1** masks IRQ; **F=1** masks FIQ. These are *disable* bits.
- **T=0** means ARM state; **T=1** means Thumb state. State and mode are separate concepts.
- **M[4:0]** selects one of seven modes. Reserved patterns are invalid.

Read CPSR with `MRS r0,CPSR`. A privileged read–modify–write through `MSR CPSR_c,r0` changes permitted control bits while preserving unrelated fields. User mode can update flags but cannot change privileged mode/mask fields. Use `BX` or exception return to change instruction state; **do not change T using MSR**. Later architecture Q/J/GE fields must not be inserted into an ARMv4T diagram.

3.4 Memory width, alignment and endianness

ARMv4T has a 32-bit address space with byte addressing. A byte is 8 bits, a halfword 16 and a word 32. In these examples words start on addresses divisible by 4 and halfwords on addresses divisible by 2. Register width does not require every memory transfer to be 32 bits.

Address	Little-endian bytes for 0x12345678	Big-endian bytes
0x1000	0x78 (least significant)	0x12 (most significant)
0x1001	0x56	0x34
0x1002	0x34	0x56
0x1003	0x12 (most significant)	0x78 (least significant)

Both aligned word loads reconstruct the same numerical word when the configured endianness matches the memory layout. Endianness describes **bytes in memory**, not reversal of bit numbering or register contents. ARM7's legacy unaligned word behaviour must not be treated as modern unrestricted unaligned access; keep all examples aligned.

3 marks: why are SP and LR banked?

Model answer. An exception handler needs its own stack and saved return address without overwriting the interrupted context's SP/LR. Mode switching selects those private copies. FIQ also banks R8–R12. Banking does not initialize stack memory or automatically save the shared working registers.

Verification and further reading: [TRM] programmer's model chapter 2; [ARCH] A2 programmer's model; [ABI] core-register and stack conventions; [FURBER] chapter 2.

4 Condition flags and conditional execution

4.1 What NZCV mean

Flags describe the last operation that *actually wrote them*, not necessarily the most recent instruction. `ADDS/SUBS` write a result and flags. `CMP/CMN/TST/TEQ` set flags and discard the result. Ordinary `ADD/SUB/MOV` preserve flags.

Flag	Meaning	Rule for arithmetic flag-setting operations
N	Negative	Equals result bit 31; alone it is not a signed comparison result.
Z	Zero	One exactly when all 32 result bits are zero.
C	Carry	Addition: carry out of bit 31. Subtraction: **no borrow**, set when unsigned first operand $\geq$ second.
V	Signed overflow	Exact signed arithmetic result lies outside $[-2^{31}, 2^{31}-1]$.

For 32-bit operands A, B, let $R = (A + B) \bmod 2^{32}$. C tests whether the full unsigned sum is at least 2^{32}. V is set if inputs have the same sign and the result has the opposite sign. For subtraction $R = (A - B) \bmod 2^{32}$, C tests $A \geq B$ unsigned; V is set if input signs differ and the result sign differs from A.

4.2 Worked flag laboratory

The following answers were recomputed using exact integers and checked against executed ARM machine code. NZCV columns are in that order.

Operation	32-bit result	NZCV	Explanation
`7-7` (CMP)	`00000000`	0110	Zero; no unsigned borrow.
`FFFFFFFF+1`	`00000000`	0110	Unsigned carry; signed $-1+1=0$ fits.
`7FFFFFFF+1`	`80000000`	1001	Positive signed overflow; no unsigned carry.
`0-1`	`FFFFFFFF`	1000	Borrow; signed -1 fits.
`80000000-1`	`7FFFFFFF`	0011	No unsigned borrow; negative signed underflow.
`80000000+80000000`	`00000000`	0111	Both unsigned carry and signed overflow.
`30-12`	`00000012`	0010	Decimal 18; no borrow or signed overflow.
`12-30`	`FFFFFFEE`	1000	Decimal -18 signed; unsigned borrow.

Worked example: Signed and unsigned can disagree

After `CMP r1,r2` with R1=FFFFFFFF, R2=1, subtraction gives `FFFFFFFE`; NZCV=1010. Unsigned R1 is greater than R2, so HI is true. Signed R1 is $-1 < 1$, so LT is true. Both statements are consistent: **the interpretation determines the condition to use**.

4.3 The complete condition-code reference

Suffix (code)	Flag test	Meaning after CMP A, B
EQ (0000)	$Z = 1$	Equal.
NE (0001)	$Z = 0$	Not equal.
CS/HS (0010)	$C = 1$	Unsigned higher or same, $A \geq B$.
CC/LO (0011)	$C = 0$	Unsigned lower, $A < B$.
MI (0100)	$N = 1$	Result bit 31 is one.
PL (0101)	$N = 0$	Result bit 31 is zero.
VS (0110)	$V = 1$	Signed overflow occurred.
VC (0111)	$V = 0$	No signed overflow.
HI (1000)	$C = 1$ and $Z = 0$	Unsigned strictly higher, $A > B$.
LS (1001)	$C = 0$ or $Z = 1$	Unsigned lower or same, $A \leq B$.
GE (1010)	$N = V$	Signed greater or equal, $A \geq B$.
LT (1011)	$N \neq V$	Signed less, $A < B$.

Suffix (code)	Flag test	Meaning after CMP A, B
GT (1100)	$Z = 0$ and $N = V$	Signed strictly greater, $A > B$.
LE (1101)	$Z = 1$ or $N \neq V$	Signed less or equal, $A \leq B$.
AL (1110)	Always	Default when suffix omitted.

Signed comparisons need V to correct misleading result sign after overflow. Use EQ/NE for equality regardless of signedness; use HI/HS/LO/LS for full-range unsigned values. BGT/BLT are appropriate for *signed* values, not an unsigned dividend merely because its classroom examples happen to be small.

4.4 Conditional instructions and stale flags

```
CMP   r1, r2
MOVGE r0, r1       @ signed maximum: r1 >= r2
MOVLT r0, r2       @ otherwise
```

The first MOV does not set flags, so the second still tests the same CMP. For an unsigned maximum use MOVHS/MOVLO. A false condition causes no destination or flag change. Conditional execution can avoid a branch for a short sequence, but it is not a universal speedup: instructions still occupy instruction slots and timing depends on the core.

Read the last flag writer

`SUB r3,r1,r2` followed by `MOVEQ r4,#1` tests *old* Z. Use SUBS or CMP to establish the intended flags. Initialize R4 to zero first if the required output is 0/1; a skipped MOVEQ does not itself write zero.

4.5 Logical, shift and multiply flags

Instruction class	Effect when flag setting is requested
AND, EOR, ORR, BIC, MOV, MVN; TST, TEQ	N/Z from result, **C from operand-2 shifter**, V unchanged. Unshifted register operand keeps C.
LSL/LSR/ASR/ROR via MOVS	N/Z from result; C is the last bit shifted out when a shift occurs; V unchanged.
MULS on ARMv4T	N/Z from low product; **C unpredictable**; V unchanged.
ADDS, ADCS, SUBS, SBCS, CMP, CMN	Arithmetic N/Z/C/V rules.

For a rotated immediate in a logical S instruction, rotation zero preserves C; nonzero rotation supplies bit 31 of the resulting immediate as C. A shift does not set V merely because significant bits are lost. MULS is not an overflow detector; use a full long product and inspect its high word.

3 marks: distinguish C and V.

Model answer. C describes unsigned carry/no-borrow; V describes overflow of signed two's-complement arithmetic. `FFFFFFFF+1` sets C but not V; `7FFFFFFF+1` sets V but not C. Both describe the same 32-bit result under different numerical interpretations.

Verification and further reading: [ARCH] CPSR flags, condition field, data-processing operands and multiply instructions; [TRM] condition-field table and programmer's model.

5 Operating modes, exceptions and mode switching

5.1 Why there are seven modes

Modes provide privilege and exception context. User is unprivileged. System is privileged but shares User registers. Five exception modes hold private SP/LR and saved status. Mode is not instruction state: User, System or a handler can conceptually use ARM or Thumb; **exception entry forces ARM state on ARM7TDMI**.

Mode	M[4:0] / hex	Private registers	How/why entered
User	10000 / 0x10	None; no SPSR	Unprivileged application code.
FIQ	10001 / 0x11	R8–R14, SPSR	Fast interrupt request; extra scratch registers.
IRQ	10010 / 0x12	R13/R14, SPSR	Normal interrupt request.
Supervisor (SVC)	10011 / 0x13	R13/R14, SPSR	Reset and software interrupt.
Abort	10111 / 0x17	R13/R14, SPSR	Prefetch/data abort.
Undefined	11011 / 0x1B	R13/R14, SPSR	Undefined instruction.
System	11111 / 0x1F	None; no SPSR	Privileged code using User bank; software mode change.

FIQ can work in R8–R12 without disturbing the interrupted non-FIQ program's registers. A handler using shared registers must save/restore them. IRQ does not automatically save R0–R12 on a stack; ARM7 exception entry differs from Cortex-M hardware stacking.

Privilege does not automatically protect every MCU register

User cannot change privileged CPSR fields. Whether User can access a peripheral or RAM region depends on memory/bus protection implemented by the system. Bare ARM7TDMI has no built-in MMU/MPU. Mode alone is not proof that a memory-mapped GPIO or interrupt-controller register is inaccessible.

5.2 The ARM7TDMI exception vectors

These are the **classic low-vector addresses**, each one ARM instruction wide. They normally contain a branch or PC-loading instruction leading to a handler, not a Cortex-M-style table of raw function pointers.

Address	Exception	Mode	Masks on entry	Usual ARM-state return
0x00	Reset	SVC	I=1, F=1	No ordinary return.
0x04	Undefined instruction	Und.	I=1, F kept	`MOVS pc,lr`

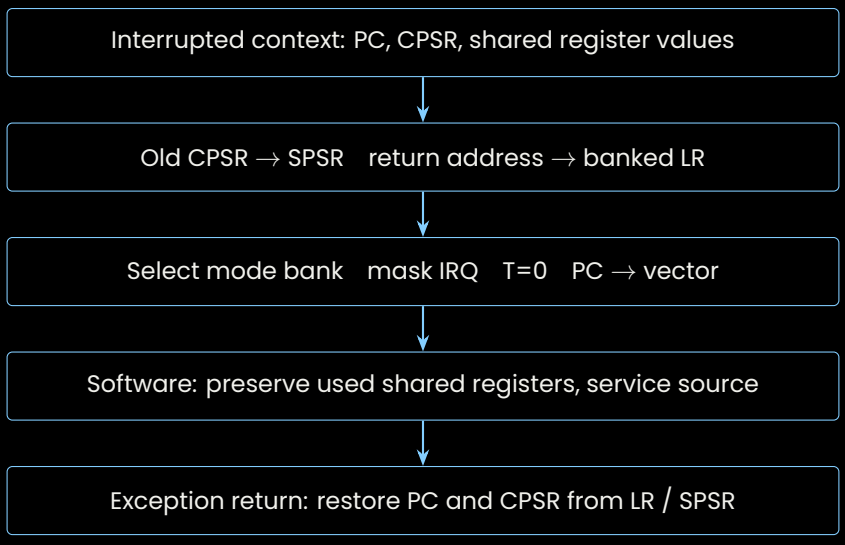
Address	Exception	Mode	Masks on entry	Usual ARM-state return
0x08	Software interrupt (SWI)	SVC	I=1, F kept	`MOVS pc,lr`
0x0C	Prefetch abort	Abort	I=1, F kept	`SUBS pc,lr,#4`
0x10	Data abort	Abort	I=1, F kept	`SUBS pc,lr,#8`
0x14	Reserved	—	—	Not an exception.
0x18	IRQ	IRQ	I=1, F kept	`SUBS pc,lr,#4`
0x1C	FIQ	FIQ	I=1, F=1	`SUBS pc,lr,#4`

"F kept" means its previous value is retained, not necessarily cleared. Return forms above show the common policy: SWI/undefined continue after the triggering instruction, prefetch/data abort retry the faulting instruction after correcting the cause, IRQ/FIQ resume the next instruction. Abort recovery policy may instead terminate or redirect the program. Reset starts privileged in ARM state with both interrupt classes masked; non-control flags must not be assumed initialized to zero.

FIQ is last, so a fast handler can begin directly at 0x1C if vector placement permits. It also has extra banked registers and higher priority than IRQ. An MCU interrupt controller selects which peripheral maps to IRQ/FIQ; the core alone does not supply a separate classic vector for every UART/timer source. A reserved vector slot may have vendor-specific boot/checksum usage, without becoming a new ARM exception.

5.3 Exception entry: what hardware saves

For an accepted exception other than reset, hardware saves the old CPSR in the destination mode's SPSR, places the exception-specific return address in that mode's LR, changes CPSR mode/mask/state, and begins ARM execution at the vector. Shared working registers and memory are **not** automatically stacked. Exact recognition of aborts or a multi-cycle instruction follows the core's exception rules; "finish the current instruction" is a useful IRQ outline, not a universal description of all faults.



Worked example: IRQ entry and return, completely traced

Assume an ARM instruction at 0x8000 completes, so the next instruction is at 0x8004. User CPSR is `60000010`: N=0, Z=1, C=1, V=0; IRQ/FIQ unmasked; T=0. A recognized IRQ produces:

1. `SPSR_irq=60000010`; `LR_irq=00008008` (next address +4).
2. CPSR becomes `60000092`: same flags, IRQ mode 0x12, I=1, F unchanged, T=0.
3. R13/R14 now refer to IRQ's private SP/LR; PC begins at vector 0x18.
4. Software services the source and restores any shared registers it changed.
5. `SUBS pc,lr,#4` loads PC=0x8004 and restores CPSR=`60000010` from SPSR_irq.

`MOV pc,lr` would neither subtract the IRQ offset nor restore CPSR. `BX lr` is a normal subroutine/state-changing return, not a replacement for this exception return.

5.4 Software mode switching and initialized stacks

A **privileged** program can explicitly change M[4:0]. It must preserve unrelated CPSR bits. A software mode write does not save CPSR into an SPSR, calculate a return LR or branch to a vector.

Complete mode-switch experiment

```
.syntax unified
.cpu arm7tdmi
.arm
.text
.balign 4
.global _start
_start:
    MRS r0, CPSR          @ start in privileged ARM state
    MOV r1, r0            @ retain starting CPSR
    BIC r0, r0, #0x1F
    ORR r0, r0, #0x1F     @ System; preserve I,F,T/flags
    MSR CPSR_c, r0
    MRS r2, CPSR          @ low five bits now 11111
    MSR CPSR_c, r1        @ return to starting mode
    MRS r3, CPSR
done: B done              @ no SPSR saved by this switch
```

With the verification harness's initial CPSR `000000D3`, R1 retains that value, R2 becomes `000000DF` (System), and R3 returns to `000000D3`. R0–R3 are shared across these modes. The experiment never writes T. In User mode a privileged control-field write cannot promote the program; a software interrupt requests privileged service instead.

Each stack pointer has to be initialized to valid writable RAM before stack instructions or interrupts use it. Startup usually runs in SVC, temporarily visits exception modes to assign their SP values, initializes the User/System stack, configures vectors/controller and then unmasks interrupts. A mode transition does not manufacture a valid stack. Do not use a banked register to preserve information that must survive switching to a different bank.

5.5 IRQ handler structure and nesting

The following is an **assembleable structural fragment**, not an operational board ISR. It assumes IRQ stack initialization, correct vector routing, and no deliberate same-mode nesting. The selected device must be serviced and acknowledged in the indicated location.

```
.syntax unified
.cpu arm7tdmi
.arm
.text
.global irq_handler
irq_handler:
    STMDB sp!, {r0-r3, r12, lr} @ 24 bytes, SP already valid
    @ Service and acknowledge the selected device HERE.
    @ This placeholder is not an operational board ISR.
    LDMIA sp!, {r0-r3, r12, lr}
    SUBS pc, lr, #4           @ restore CPSR from SPSR_irq
```

It saves shared R0–R3 and R12 plus IRQ LR on a full descending stack. Six registers occupy 24 bytes, preserving an initially 8-byte-aligned SP. SPSR is left in hardware because nesting is disabled. Before enabling nested IRQs, save SPSR_irq and the return LR safely: another IRQ can overwrite them. FIQ can preempt an IRQ if F is clear; a FIQ handler still saves shared registers it uses and preserves any state required for deeper nesting.

Typical exception priority from highest to lowest on ARM7TDMI is reset, data abort, FIQ, IRQ, prefetch abort, undefined instruction/software interrupt (mutually exclusive decode outcomes). Priority and masking are separate: a high-priority event must still satisfy recognition/mask rules. Keep this core exception hierarchy separate from a vendor interrupt controller's peripheral priorities.

3 marks: distinguish mode switching by MSR from an exception.

Model answer. MSR changes permitted CPSR fields from privileged code and selects banks; it does not automatically save SPSR/LR or jump to a vector. An accepted exception saves old CPSR/return information into its mode bank, changes control state and redirects execution to the vector. User mode requests privilege through an exception, not a CPSR mode write.

Verification and further reading: [TRM] chapter 2, especially exception entry/exit summary and exception priority; [ARCH] programmer's model and MRS/MSR; [NXP] device-level interrupt integration; [FURBER] chapters 2, 5 and 11.

6 Writing and tracing real ARM assembly

6.1 Read an instruction before memorizing it

```
loop: ADD r0, r1, r2, LSL #2  @ r0 = r1 + 4*r2
```

`loop` names the address; ADD is the mnemonic; R0 is destination; R1 is the first source; shifted R2 is operand 2; the comment is for the reader. `MOV r0,r1` copies R1, whereas `MOV r0,#1` uses the constant 1. S means **set flags**, not signed. Conditions are optional: `ADDEQ` executes when Z=1. Under GNU unified syntax a combined suffix can be written `ADDSEQ` (S then condition); legacy syntax may write `ADDEQS`. Use the chosen assembler's accepted spelling consistently.

Instruction group	Meaning and example
Data processing	`ADD r0,r1,r2`; MOV/MVN, arithmetic, logic and comparisons use register/immediate operands.
Multiply	`MUL r0,r1,r2`; distinct encoding; no immediate operand.
Data transfer	`LDR r0,[r1]`, `STR r0,[r1]`; includes byte/halfword/block forms.
Branch/control	`B loop`, `BL function`, `BX lr`; status operations MRS/MSR and SWI.

These are useful teaching groups, not the entire architecture. Branch targets are **labels/addresses**, register-indirect branching uses BX, and arithmetic destinations are generally registers. Most examples avoid PC as a data register.

6.2 Source directives and the complete-program wrapper

All `code/*.s` examples contain an explicit GNU wrapper selecting ARM7TDMI and ARM state. `.text` selects code, `.balign 4` aligns it, `.global _start` exports an entry symbol. Labels end with a colon; `@` introduces a GNU ARM comment. Directives guide assembly/linking and do not necessarily become executable instructions.

GNU source used here	Traditional Keil armasm counterpart
`.cpu arm7tdmi; .arm`	Select CPU in project/assembler; `ARM` state directive.
`.text`, `.data`	`AREA name,CODE,READONLY` or data AREA.
`.global _start`	`EXPORT` entry symbol; `ENTRY` where appropriate.
`.word 10,20`	`DCD 10,20`.
`.ltorg`	`LTORG`; literal-pool placement.
`@ comment`, `label:`	`; comment`; traditional labels use assembler-specific layout.

Mnemonic ideas transfer, but the whole source file cannot be pasted unchanged into every assembler. `END` in armasm is a directive terminating source, not a CPU instruction. `HALT/STOP` in the classroom engine are teaching commands. A real ARM7 example uses a terminal branch loop, a return to a caller or a documented debugger mechanism.

6.3 How to assemble and inspect an example

From the source folder, with GNU Arm binutils installed:

```
arm-none-eabi-as -mcpu=arm7tdmi --fatal-warnings \
  -o build/add.o code/add.s
arm-none-eabi-ld -T code/example.ld \
  -o build/add.elf build/add.o
arm-none-eabi-objdump -d build/add.elf
```

The linker places code at 0x10000 **for the test harness**. The debugger/emulator begins at `_start`, maps code/data/RAM and stops before executing the terminal `done` loop. `factorial_call.s` uses a harness stack at 0x2F000. These addresses are not an LPC2148 startup/stack prescription. A board build needs its real memory map, reset vector, startup, stack layout and hardware initialization.

6.4 Immediate constants and literal loads

An ordinary A32 data-processing immediate is

$$\mathrm{ROR}_{32}(\text{8-bit value},\ 2 \times \text{4-bit rotation}).$$

Not every 32-bit constant fits. `#0xFF000000` is 0xFF rotated right by 8, so it is encodable. `#0x12345678` is not. `MVN r0,#0` efficiently constructs all ones. `LDR r0,=0x12345678` is an **assembler pseudo-instruction**: the assembler may use MOV/MVN if possible or a PC-relative load from a nearby literal pool. A pseudo-instruction is different from an illegal simulator mnemonic.

```
LDR r0, =0x12345678
B   done                @ branch over embedded literal data
.ltorg                  @ place literal pool within reach
done: B done
```

For an ARM-state literal LDR, the PC base is instruction address $+8$ and the ordinary immediate offset is at most 4095 bytes. Keep pools within range and outside normal fall-through execution. ARM7TDMI has no MOVW/MOVT; those modern constant-loading instructions are not a substitute on this core.

6.5 Shifts, rotates and the barrel shifter

Form	Operation	Numerical interpretation
LSL	Shift left, zeros enter low bits.	Multiply by 2^k modulo 2^{32}.
LSR	Shift right, zeros enter high bits.	Unsigned division by 2^k, truncating remainder.
ASR	Shift right, copies sign bit.	Signed division rounding toward $-\infty$, not generally toward zero.
ROR	Rotate right, low bits wrap high.	Bit rearrangement; no arithmetic division interpretation.
RRX	One-bit rotate through carry.	Old C enters bit 31; old bit 0 becomes carry when flags are set.

`MOV r0,r1,LSL #3` and the standalone GNU alias `LSL r0,r1,#3` represent a shifted MOV in ARM state. `MOVS r0,r1,LSR #1` sets C to the old bit 0; `ADC` can consume it. `MOV r0,r1,ASR #1` on -7 gives -4, while signed division truncating toward zero gives -3.

The edge cases are architectural rules

Immediate LSL accepts 0–31; LSR/ASR accept 1–32. ROR accepts 1–31, with RRX representing its special zero-encoded case. For register-controlled shifts only the low byte of the count is used. A count of zero leaves result and shifter carry unchanged. LSL by 32 gives zero and carry=old bit 0; LSR by 32 gives zero and carry=old bit 31. Larger LSL/LSR counts give zero with carry zero. ASR at 32 or more yields all copies of the sign bit. JavaScript's masked shift counts are not a hardware oracle.

A register-controlled shift has an extra internal cycle on ARM7. Immediate shifts folded into a normal ALU operand need no separate instruction; neither statement guarantees a fixed board-level time in slow memory.

6.6 Branches, calls and a stack

`B` changes control flow; `BL` also writes LR to the next instruction address. At ARM address A, BL sets LR=$A+4$ even though the branch displacement uses PC=$A+8$. A leaf subroutine can return with `BX lr`, which supports ARM/Thumb interworking. `MOV pc,lr` works for a simple same-state ARM return but does not switch state based on LR bit 0.

Worked example: Compute a branch displacement

An ARM `B` at 0x8000 targeting 0x8020 uses $(0x8020 - 0x8008)/4 = 6$. Its signed 24-bit word displacement is 6; the machine word is `EA000006`. The target must be ARM aligned. BL uses the same displacement form but also updates LR.

A function that calls another must preserve its own return LR first. `STMDB sp!,{r4,lr}` decrements SP and stores registers; `LDMIA sp!,{r4,lr}` restores them and advances SP. "!" requests writeback. A full descending stack grows to lower addresses. Under the common AAPCS convention, preserve R4–R8, R10/R11 and SP; R9 is platform-defined. R0–R3/R12 and flags are caller-clobbered; keep SP 8-byte aligned at public interfaces. The complete factorial subroutine illustrates this.

6.7 Loads/stores and indexed addressing

Instruction	Address and pointer behaviour
`LDR r0,[r1]`	Load word at R1; R1 unchanged.
`LDR r0,[r1,#4]`	Load at R1+4; R1 unchanged.
`LDR r0,[r1,#4]!`	Load at R1+4; write R1+4 back to R1.
`LDR r0,[r1],#4`	Load at old R1; then R1 becomes old R1+4.
`LDRB r0,[r1]`	Load one byte, zero-extend it.
`STR r0,[r1]`	Store the word value in R0 at address R1.

`LDR r3,=values` loads an *address*; `LDR r1,[r3]` reads the memory at that address. The following complete program sums four aligned words. For each iteration the pointer advances by 4 because a word occupies four byte addresses.

Sum an array in memory

```
.syntax unified
.cpu arm7tdmi
.arm
.text
.balign 4
.global _start
_start:
    LDR r3, =values       @ array address, not its contents
    MOV r2, #4            @ number of words
compute:
    MOV r0, #0
    CMP r2, #0
    BEQ done
loop:
    LDR r1, [r3], #4      @ load a word then advance pointer
    ADD r0, r0, r1
    SUBS r2, r2, #1
    BNE loop
done: B done              @ r0=10+20+30+40=100
.ltorg
.data
.balign 4
values: .word 10, 20, 30, 40
```

The running sum is 10, 30, 60, 100; the count becomes 3, 2, 1, 0. S on SUBS supplies loop flags; ADD leaves them alone. The initial zero-count check makes the loop safe if a debugger changes R2 to zero. This is a static, real-memory counterpart to the classroom RISC array demonstration.

3 marks: distinguish MOV, LDR and LDR pseudo-instruction.

Model answer. MOV copies a register or an encodable immediate. `LDR r0,[r1]` loads a word from memory addressed by R1. `LDR r0,=constant` asks the assembler to construct/load a constant, potentially using a literal pool; it is not the same source-level operation as loading through R1.

Verification and further reading: [GNU] directives and ARM syntax; [ASM] constant loading and stack chapters; [ARCH] shifts, branch and addressing rules; [ABI] register/stack convention; [FURBER] chapters 3, 5 and 6.

7 Arithmetic ALPs: design, code and results

7.1 Addition and the register plan

For each ALP state the inputs, outputs, register roles and what may be overwritten. Here R1/R2 are inputs and R0 is the output; R1/R2 remain unchanged. ADD keeps only 32 result bits. ADDS also sets NZCV, letting software distinguish unsigned carry from signed overflow.

Add two numbers and set flags

```
.syntax unified
.cpu arm7tdmi
.arm
.text
.balign 4
.global _start
_start:
    MOV r1, #25
    MOV r2, #17
compute:
    ADDS r0, r1, r2       @ sum; update NZCV
done: B done             @ inspect r0 and CPSR here
```

The complete program computes $25 + 17 = 42$ (`0000002A`); NZCV=0000. At `compute`, only ADDS writes R0 and flags; the terminal branch preserves the observable result. Loading operands with MOV without S does not deliberately reset flags. ADDS establishes all four arithmetic flags regardless of their starting values.

Worked example: Carry versus overflow in addition

$0xFFFFFFFF + 1 = 0x1\,00000000$ as an exact unsigned sum. The low word is zero, C=1, Z=1, N=0, V=0. For signed interpretation it is $-1 + 1 = 0$, which fits. By contrast $0x7FFFFFFF + 1$ produces `80000000`: C=0, V=1. The S suffix exposes these conditions; ordinary ADD has the same low result but leaves old flags unchanged.

7.2 Subtraction, reverse subtraction and borrow

Subtract and interpret no-borrow

```
.syntax unified
.cpu arm7tdmi
.arm
.text
.balign 4
.global _start
_start:
    MOV r1, #30
    MOV r2, #12
compute:
    SUBS r0, r1, r2       @ C = 1 means no borrow
done: B done
```

It produces decimal 18, `00000012`, NZCV=0010. Swapping operands produces $12 - 30 = -18$, represented as `FFFFFFEE`, with NZCV=1000. SUB is $Rn -$ operand2; RSB reverses that order. `RSB r0,r1,#0` computes $-r_1$ modulo 2^{32}. CMP performs the subtraction solely to establish flags.

Carry-consuming arithmetic

`ADC Rd,Rn,Op2` computes $Rn + Op2 + C$. `SBC Rd,Rn,Op2` computes $Rn - Op2 - (1 - C)$. Addition propagates carry; subtraction propagates *borrow through the inverted carry convention*. Use

ADDS/SUBS on the low word before ADC/SBC on the high word. Do not let an unrelated flag-setting instruction intervene.

7.3 Two-word addition and subtraction

A 64-bit value consists of a high and a low 32-bit word. The notation high:low makes the register order explicit. These are extensions of the simple ALPs, useful for understanding carry chaining.

Add two 64-bit values

```
.syntax unified
.cpu arm7tdmi
.arm
.text
.balign 4
.global _start
_start:
    MVN r1, #0            @ first low word FFFFFFFF
    MOV r2, #1            @ first high word
    MOV r3, #1            @ second low word
    MOV r4, #2            @ second high word
compute:
    ADDS r0, r1, r3       @ low word sets carry
    ADCS r5, r2, r4       @ high word consumes carry
done: B done              @ r5:r0 = 00000004:00000000
```

Worked example: Trace the low-word carry

Inputs are `00000001:FFFFFFFF` and `00000002:00000001`. The low ADDS produces `00000000` with C=1. The high ADCS computes $1 + 2 + 1 = 4$, so the complete output is `00000004:00000000`. High-word ADCS flags describe that operation: its Z is not a test that the *entire* 64-bit result is zero. Test both words for that purpose.

Subtract two 64-bit values

```
.syntax unified
.cpu arm7tdmi
.arm
.text
.balign 4
.global _start
_start:
    MOV r1, #0            @ first low word
    MOV r2, #2            @ first high word
    MOV r3, #1            @ second low word
    MOV r4, #1            @ second high word
compute:
    SUBS r0, r1, r3       @ low word: C=0 means borrow
    SBCS r5, r2, r4       @ high subtracts (1-C)
done: B done              @ r5:r0 = 00000000:FFFFFFFF
```

The low word computes $0 - 1 =$ `FFFFFFFF` with C=0. The high SBCS computes $2 - 1 - (1 - 0) = 0$. Thus `00000002:00000000` minus `00000001:00000001` gives `00000000:FFFFFFFF`. A common incorrect answer subtracts C itself and misses the borrowed one.

7.4 Multiplication and its ARM7 restrictions

`MUL Rd,Rm,Rs` writes the low 32 bits of $Rm \times Rs$. Inputs must be registers, and PC is not a valid operand. For portable classic ARM7-style source, choose Rd different from Rm. Early architectural descriptions imposed that restriction; DDI 0100I notes that later implementations can relax it and ARMv6 does not require it. A tool can also swap commutative inputs to avoid an encoded overlap. All examples here use the conservative form explicitly, matching the classroom convention without relying on a relaxation.

Multiply two numbers

```
.syntax unified
.cpu arm7tdmi
.arm
.text
.balign 4
.global _start
_start:
    MOV r1, #12
    MOV r2, #9
compute:
    MUL r0, r1, r2        @ r0 differs from r1 (Rm)
done: B done             @ low 32 bits; flags unchanged
```

$12 \times 9 = 108$ (`0000006C`); plain MUL leaves NZCV unchanged. The low product is identical for signed and unsigned interpretations modulo 2^{32}. If the upper word matters, select UMULL for an unsigned full product or SMULL for a signed one. MULS changes N/Z, leaves V alone, and gives unpredictable C on ARMv4T; it is not a product-overflow test.

Multiply by repeated addition

```
.syntax unified
.cpu arm7tdmi
.arm
.text
.balign 4
.global _start
_start:
    MOV r1, #12           @ multiplicand
    MOV r2, #7            @ unsigned multiplier
compute:
    MOV r0, #0            @ sum identity
loop:
    CMP r2, #0
    BEQ done              @ handles multiplier zero
    ADD r0, r0, r1
    SUB r2, r2, #1
    B loop
done: B done              @ r0=84; r2=0
```

The running sum starts at 0; after each pass it has one more copy of 12. With multiplier 7, successive sums are 12, 24, 36, 48, 60, 72, 84. Testing count=0 *before* the body handles a zero multiplier. For this routine R2 is an **unsigned nonnegative iteration count**; treating a negative two's-complement value as a count would make an enormous loop. Arithmetic wraps if the product exceeds one word.

7.5 A full 64-bit product and constant multiplication

Use UMULL to preserve the full product

```
.syntax unified
.cpu arm7tdmi
.arm
.text
.balign 4
.global _start
_start:
    LDR r1, =100000
    LDR r2, =100000
compute:
    UMULL r0, r3, r1, r2  @ r3:r0 full unsigned product
done: B done              @ 00000002:540BE400
.ltorg
```

Worked example: The classroom 100000-times-100000 experiment

$100000^2 = 10,000,000,000 = 2 \times 2^{32} + 1,410,065,408$. Hence high:low is `00000002:540BE400`. MUL would return only `540BE400`; UMULL returns R3=2 and R0=`540BE400`. A nonzero high word means the unsigned product does not fit 32 bits. This is why the experiment uses literal loading rather than an unrestricted classroom immediate.

For a signed product, SMULL produces a 64-bit two's-complement result; it fits a signed 32-bit word only if the high word equals the sign extension of the low word. Long multiply also has register restrictions; choose distinct destination registers and keep them separate from the multiply input registers in beginner examples.

Multiplication by a power of two is a shift. For $10x = (8x) + (2x)$:

```
MOV r2, r1, LSL #3
ADD r0, r2, r1, LSL #1
```

For $x = 12$ the intermediate is 96 and the result is 120. This is a useful expression of the barrel shifter, not proof that all constants or workloads should avoid the multiplier. ARM7 multiply timing is data-dependent; consult instruction-cycle tables rather than claiming MUL always takes one cycle.

3 marks: explain 64-bit addition.

Model answer. Split each number into high/low words. ADDS adds low words and establishes C; ADC or ADCS adds high words plus that carry. State which output register holds each word. Final high-word Z alone does not establish whether the full 64-bit result is zero.

Verification and further reading: [ARCH] ADD/ADC/SUB/SBC/MUL/UMULL/SMULL; [TRM] instruction summaries and cycle timing; [FURBER] chapters 3 and 5.

8 Logical ALPs and bit manipulation

8.1 Bitwise operations and masks

Each output bit depends on the corresponding input bits, unlike arithmetic carry propagation. A mask selects positions: bit k has mask $1 \ll k$. Registers are 32 bits even when a classroom display shows only the low eight.

Instruction	Rule	Typical use
AND	$A \wedge B$	Keep only selected bits.
ORR	$A \vee B$	Set selected bits, preserve others.
EOR	$A \oplus B$	Toggle selected bits.
BIC	$A \wedge \neg B$	Clear selected bits, preserve others.
MVN	$\neg B$	Complement all 32 bits.
TST	AND, flags only	Test selected bits without storing a result.
TEQ	EOR, flags only	Compare bit patterns without storing a result.

For $A = \texttt{D6}$ and $B = \texttt{78}$, low-byte outputs are AND=`50`, ORR=`FE`, EOR=`AE`, BIC=`86`. Full-word MVN of `000000D6` gives `FFFFFF29`, not just `29`.

8.2 Set, clear, toggle and extract

Complete classroom logical experiment

```
.syntax unified
.cpu arm7tdmi
.arm
.text
.balign 4
.global _start
_start:
    MOV r0, #0xAA         @ 10101010
compute:
    ORR r0, r0, #1        @ AA -> AB, set bit 0
    BIC r0, r0, #0x80     @ AB -> 2B, clear bit 7
    EOR r0, r0, #0x10     @ 2B -> 3B, toggle bit 4
    AND r1, r0, #0x0F     @ r1 = 0B; r0 remains 3B
done: B done
```

Step	Binary low byte	Hex
Initial	10101010	AA
ORR bit 0	10101011	AB
BIC bit 7	00101011	2B
EOR bit 4	00111011	3B
AND low nibble into R1	00001011	0B

To test bit 5: `TST r0,#0x20`, then BEQ means clear and BNE means set. TST leaves R0 intact. `AND r0,r0,#0x20` instead destroys all other bits. `EOR r0,r0,r0` clears a register without an immediate and, without S, leaves old flags alone.

Worked example: Clear one bit without damaging the rest

R0=`D6`=11010110. Clearing bit 2 uses BIC mask `04`, giving `D2`=11010010. Clearing bit 6 uses mask `40`, giving `96`=10010110. AND with mask `40` would keep only bit 6, producing `40`, rather than preserve the other bits.

8.3 Memory-mapped peripheral application

```
LDR r1, =PORT_ADDRESS   @ symbolic, device-specific address
LDR r0, [r1]
ORR r0, r0, #0x08       @ set bit 3 in a local copy
STR r0, [r1]
```

This is a **schematic pattern**, not a complete device driver; `PORT_ADDRESS` must be defined from the selected MCU manual. It works only for a register whose documented read/write semantics permit read–modify–write. A hardware set/clear register may be better. An interrupt or another bus master can race between LDR and STR; write-one-to-clear/status registers often cannot use this generic pattern. Logical instructions operate on the core register; memory access still uses a transfer instruction.

8.4 Count set bits through the shifter carry

Population count of a 32-bit word

```
.syntax unified
.cpu arm7tdmi
.arm
.text
.balign 4
.global _start
_start:
    MOV r1, #0xCA         @ 11001010 has four set bits
compute:
    MOV r0, #0
    MOV r2, #32
loop:
    MOVS r1, r1, LSR #1   @ original low bit -> C
    ADC r0, r0, #0        @ add that bit, keep old flags
    SUBS r2, r2, #1       @ set loop flags afterwards
    BNE loop
done: B done              @ r0=4, r1=0, r2=0
```

Every iteration shifts R1 right one bit. MOVS writes the discarded low bit into C; ADC adds that bit to R0. SUBS then updates flags for the loop count, *after* ADC has consumed C. After 32 iterations all bits have been considered. `CA`=11001010 has four set bits; `FFFFFFFF` has 32; zero has none. The classroom engine does not update shifter C correctly, so its popcount execution is not an authority for this program.

Preserve the carry until its consumer

Putting `SUBS r2,r2,#1` between MOVS and ADC replaces the bit carry with the subtraction's no-borrow flag. ADC then counts the wrong thing. Conversely plain SUB would not establish Z for BNE; ordering and the S suffix both matter.

8.5 Reverse four bits and find a GCD

Complete the classroom four-bit reversal challenge

```
.syntax unified
.cpu arm7tdmi
.arm
.text
.balign 4
.global _start
_start:
    MOV r0, #0xD          @ low nibble 1101
compute:
    AND r1, r0, #0xF      @ ignore upper 28 bits
    MOV r0, #0
    MOV r2, #4
loop:
    MOVS r1, r1, LSR #1
    ADC r0, r0, r0        @ append low bit to result
    SUBS r2, r2, #1
    BNE loop
done: B done              @ r0=0xB (1011)
```

Masking first ignores the upper 28 bits. Four iterations remove low bits one at a time and append them to the result using $2r_0 + C$. For input low nibble 1101 the removed bits are 1,0,1,1 and the result is 1011 (B). This reverses bit order; it does not complement bits or reverse byte order.

Unsigned Euclid by repeated subtraction

```
.syntax unified
.cpu arm7tdmi
.arm
.text
.balign 4
.global _start
_start:
    MOV r1, #48           @ unsigned a
    MOV r2, #18           @ unsigned b
compute:
    CMP r1, #0
    MOVEQ r0, r2
    BEQ done              @ gcd(0,b)=b
    CMP r2, #0
    MOVEQ r0, r1
    BEQ done              @ gcd(a,0)=a, including (0,0)=0
loop:
    CMP r1, r2
    BEQ equal
    SUBHI r1, r1, r2      @ no S: preserve CMP flags
    SUBLO r2, r2, r1
    B loop
equal:
    MOV r0, r1
done: B done              @ r0=6
```

Zero inputs are handled before the loop. For positive values, subtracting the smaller from the larger preserves the GCD and reduces their sum, so the loop terminates when they are equal. For 48 and 18: $(48, 18) \to (30, 18) \to (12, 18) \to (12, 6) \to (6, 6)$, giving 6. The conditional SUBHI/SUBLO do not set flags, so both test the same CMP. Using SUBHIS would change flags before SUBLO and break that reasoning. This algorithm is simple but can be slow; modulo-based Euclid is an extension.

3 marks: give the set/clear/toggle idioms.

Model answer. For bit k, form mask $1 \ll k$. ORR with the mask sets the bit; BIC clears it; EOR toggles it. Each preserves unselected bits. AND with a mask extracts selected bits instead. A memory-mapped register still needs documented transfer and concurrency handling.

Verification and further reading: [ARCH] logical operations, shifter carry and conditional execution; [FURBER] chapters 3 and 5.

9 Division without a hardware divider

9.1 Specify the problem and its boundary cases

ARMv4T/ARM7TDMI provides no DIV, UDIV or SDIV instruction. Software builds division from comparisons, shifts and subtraction. Some later architectures, including Cortex-M3's ARMv7-M, provide divide; Cortex-M0/M0+ do not. The absence is a property of the selected core, not every ARM processor.

For unsigned dividend A and positive divisor B, compute quotient Q and remainder R satisfying

$$A = QB + R, \qquad 0 \le R < B.$$

Check $B = 0$ first. These examples report an error status rather than loop forever. They do not implement signed division; interpreting negative bit patterns as unsigned changes the problem.

9.2 Repeated subtraction: the clearest first ALP

Unsigned divide with a zero-divisor guard

```
.syntax unified
.cpu arm7tdmi
.arm
.text
.balign 4
.global _start
_start:
    MOV r1, #47           @ unsigned dividend
    MOV r2, #6            @ unsigned divisor
compute:
    MOV r0, #0            @ quotient
    MOV r5, #0            @ status: zero means success
    CMP r2, #0
    BEQ zero_divisor
loop:
    CMP r1, r2
    BLO done              @ unsigned remainder < divisor
    SUB r1, r1, r2
    ADD r0, r0, #1
    B loop
zero_divisor:
    MOV r5, #1            @ error; dividend unchanged
done: B done              @ r0=7, r1=5, r5=0
```

Register plan: R1 starts as dividend and becomes remainder, R2 holds divisor, R0 counts quotient, R5 is status. Start Q=0. While $R \ge B$ unsigned, reduce R by B and increase Q. The invariant $A = QB + R$ is maintained. Exit only when $R < B$; equality still permits one valid subtraction. BLO and BCC are aliases testing unsigned lower; BLT tests signed less and is unsafe for full-range unsigned operands.

Completed subtractions	0	1	2	3	4	5	6	7
R1 remainder	47	41	35	29	23	17	11	5
R0 quotient	0	1	2	3	4	5	6	7

The next comparison finds $5 < 6$, so the output is Q=7, R=5. For 48/6, subtraction must continue when the remainder equals 6; the output is 8,0. A BLE/BLS exit would stop early on that equality. For the supplied debugging input 47/6, replacing BLT with BLE does *not* change the small-value result; the actual defect needs an exact-division test to expose it.

Correctness is not a speed guarantee

The loop executes Q subtractions. Dividing FFFFFFFF by 1 would require over four billion iterations. Use repeated subtraction for understanding and small bounded operands; a fixed-bit division algorithm supports full-width data more practically.

9.3 Binary shift-and-subtract, first in eight bits

Process dividend bits from most significant to least significant. Start partial remainder $r = 0$ and quotient $q = 0$. At each step append the next dividend bit to r, shift q left, and if $r \geq B$, subtract B and append a quotient 1; otherwise append 0. After each step r is below B. This is binary long division (a restoring-style compare/subtract algorithm).

Worked example: 23 divided by 5: static version of the animation

$23 = 00010111_2$. The eight-step trace is:

Step	Input bit	Trial $2r + b$	Subtract 5?	New r	New q
1	0	0	No	0	0
2	0	0	No	0	0
3	0	0	No	0	0
4	1	1	No	1	0
5	0	2	No	2	0
6	1	5	Yes	0	1
7	1	1	No	1	2
8	1	3	No	3	4

Thus Q=4, R=3 and $23 = 4(5) + 3$. Eight iterations apply to an **eight-bit dividend**. Merely changing a classroom eight-bit counter to 32 without reviewing extraction and overflow is not a complete full-width algorithm.

9.4 A complete 32-bit routine with the transient carry

Full-width unsigned division

```
.syntax unified
.cpu arm7tdmi
.arm
.text
.balign 4
.global _start
_start:
    MOV r1, #23           @ unsigned dividend
    MOV r2, #5            @ unsigned divisor
compute:
    MOV r0, #0            @ quotient
    MOV r3, #0            @ partial remainder
    MOV r4, #32           @ process all 32 dividend bits
    MOV r5, #0            @ status
    CMP r2, #0
    BEQ zero_divisor
loop:
    MOV r0, r0, LSL #1    @ space for next quotient bit
    MOVS r1, r1, LSL #1   @ next dividend bit -> C
    ADCS r3, r3, r3       @ 2*remainder + input carry
    BCS reduce            @ 33rd bit implies divisor fits
    CMP r3, r2
    BLO next_bit          @ 32-bit remainder below divisor
reduce:
    SUB r3, r3, r2        @ reduce once, without flags
    ORR r0, r0, #1        @ quotient bit is one
next_bit:
    SUBS r4, r4, #1
    BNE loop
    MOV r1, r3            @ return final remainder in r1
    B done
zero_divisor:
    MOV r5, #1            @ r1 remains original dividend
done: B done              @ r0=4, r1=3, r5=0
```

R0 is quotient, R1 is the shifting dividend and finally remainder, R2 is divisor, R3 is partial remainder, R4 is a 32-step count and R5 is status. MOVS shifts the next dividend bit into C; ADCS forms $2r + C$ and exposes a possible **33rd bit**. If that carry is set, the full trial remainder is certainly at least the 32-bit divisor, so subtract once. Otherwise compare normally using BLO. The final subtraction's low-word arithmetic is correct even when the trial remainder briefly needed 33 bits.

Why one subtraction is sufficient

Before a step, $0 \le r < B$. Therefore $0 \le 2r+b \le 2B-1$ for bit $b \in \{0,1\}$. One subtraction when the trial value is at least B restores $0 \le r < B$. If B is large, the trial value can exceed $2^{32} - 1$; ADCS carry preserves the missing information. Ignoring that carry can give wrong full-width answers even if 23/5 works.

The routine executes **32 iterations for every nonzero divisor**, including zero dividend. That is a fixed iteration count, **not fixed cycle count**: branches and memory/instruction timing still vary.

Official library routines may use different optimized algorithms; do not infer their exact implementation from this classroom version.

9.5 Power-of-two division and signed rounding

For unsigned A, dividing by 2^k is `MOV r0,r1,LSR #k`. The remainder is the low k bits, obtained by masking if needed. $23/8$ gives Q=2 and R=7. A shift is not correct general division by an arbitrary divisor.

For signed negative values ASR rounds down: $-7\,\mathrm{ASR}\,1 = -4$. If a signed division contract truncates toward zero, $-7/2 = -3$. One can apply an appropriate bias before shifting or call a signed division routine; state the rounding rule. Signed full-width division also needs a policy for the overflow case $-2^{31}/-1$. This is beyond the unsigned ALP contract rather than an omitted guard inside it.

3 marks: explain software division on ARM7TDMI.

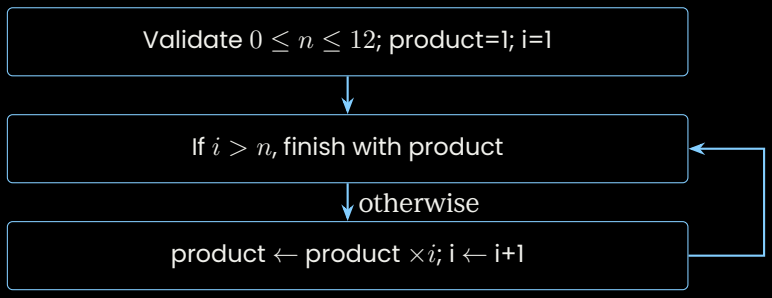
Model answer. It has no divide instruction. Repeated subtraction counts how often the divisor fits, but takes quotient-dependent time. Binary shift-and-subtract produces one quotient bit per step, using 32 iterations for a 32-bit dividend. Both require a nonzero-divisor check; unsigned conditions and transient remainder carry matter for full-width correctness.

Verification and further reading: [TRM] ARMv4T instruction summary; [ARCH] arithmetic/shifter flags; [PROFILE] architecture comparison; algorithms independently derived and machine-code tested.

10 Factorial: loops, bounds and a subroutine

10.1 Translate the algorithm into a register plan

For a nonnegative integer n, $n! = 1 \cdot 2 \cdots n$; **0!=1**. Product starts at 1 (the multiplicative identity). Let R1 hold n, R0 hold the product, R2 hold counter i, R3 be multiply scratch and R4 report status.



At the start of an iteration, product=$(i-1)!$. If $i \le n$, multiplication establishes $i!$ and increasing i restores the invariant. When $i = n+1$, product is n!. For n=0, the loop is skipped and the initialized 1 is already correct.

10.2 Complete bounded factorial program

Compute n! and reject values outside 0–12

```
.syntax unified
.cpu arm7tdmi
.arm
.text
.balign 4
.global _start
_start:
    MOV r1, #6            @ unsigned n
compute:
    MOV r0, #1            @ product identity, including 0!
    MOV r2, #1            @ counter i
    MOV r4, #0            @ status
    CMP r1, #12
    BHI out_of_range      @ reject overflow and negatives
loop:
    CMP r2, r1
    BHI done              @ exit only when i > n
    MUL r3, r0, r2        @ Rd=r3 differs from Rm=r0
    MOV r0, r3
    ADD r2, r2, #1
    B loop
out_of_range:
    MOV r0, #0            @ invalid-result sentinel
    MOV r4, #1
done: B done              @ n=6: r0=720, r4=0
```

The unsigned range check rejects both 13 and negative two's-complement bit patterns. BHI after comparing i and n exits only when i is strictly greater; equality must still perform the final multiply. MUL writes R3, keeping its destination separate from Rm=R0. R4=0 indicates success; R4=1 and R0=0 report out-of-range input. Without that status, zero would be ambiguous as an error sentinel in more general arithmetic APIs.

Worked example: Trace n=6

i (R2)	Product before	Multiply by	Product after (R0)	Next i
1	1	1	1	2
2	1	2	2	3
3	2	3	6	4
4	6	4	24	5
5	24	5	120	6
6	120	6	720	7

At i=7 the comparison exits. R0=720=`000002D0`, R1=6, R2=7 and R4=0. Starting the product at zero would yield zero on every iteration. Omitting the counter update would repeatedly multiply by one and never reach the exit.

10.3 Why 12 is the largest safe one-word factorial

Unsigned 32-bit maximum is 4,294,967,295. $12! = 479,001,600$ fits; $13! = 6,227,020,800$ does not. The signed maximum 2,147,483,647 also lies between 12! and 13!, so the same n limit applies to a signed positive one-word result. Unchecked 13! wraps to $6,227,020,800 - 4,294,967,296 = 1,932,053,504$, a plausible positive wrong answer. N or V from an unrelated CMP/MULS cannot reliably diagnose this product overflow.

The classroom phrase "BLT instead of BGT gives (n-1)!" is not generally correct: for the shown top-tested loop, BLT exits immediately for typical n>1. **BGE instead of BGT** after comparing i,n exits at i=n and actually skips the last multiplication. In the unsigned version, BHS instead of BHI creates the equivalent off-by-one error.

10.4 A callable factorial with a saved stack frame

Complete caller and factorial subroutine

```
.syntax unified
.cpu arm7tdmi
.arm
.text
.balign 4
.global _start
_start:
    LDR sp, =0x0002F000   @ harness maps writable RAM
    MOV r0, #5            @ function argument n
    BL factorial_fn
done: B done              @ result r0=120, status r1=0
factorial_fn:
    STMDB sp!, {r4, lr}   @ 8-byte frame; preserve r4/LR
    MOV r2, r0            @ keep n in scratch register
    MOV r0, #1
    MOV r1, #0            @ success status
    CMP r2, #12
    BHI invalid
    MOV r4, #1
fact_loop:
    CMP r4, r2
    BHI fact_return
    MUL r3, r0, r4        @ legal ARM7 destination
    MOV r0, r3
    ADD r4, r4, #1
    B fact_loop
invalid:
    MOV r0, #0
    MOV r1, #1
fact_return:
    LDMIA sp!, {r4, lr}
    BX lr                @ safe ARM/Thumb-aware return
.ltorg
```

The caller passes n in R0 and BL sets LR. The function saves R4 and LR in an 8-byte frame, copies n to scratch R2 and returns the result in R0, status in R1. Its internal loop uses R4 as counter and restores R4 before BX LR. R0–R3 and flags are caller-clobbered. SP returns to its entry value. This

has a documented result/status convention using ABI result registers; it is an assembly interface, not a claim that it directly implements an unspecified C signature.

The test harness maps the specified stack RAM; physical firmware must replace that address using its linker/board memory map. A leaf could avoid saving LR if it never calls another function, but keeping a clear frame makes the preserve/restore rule visible. If a function later adds a nested BL, failing to preserve the incoming LR loses its caller's return address.

3 marks: explain factorial initialization and overflow.

Model answer. Start the product at 1 so the first multiplication is correct and 0! is handled. Multiply counters through n inclusively, then exit. A 32-bit output can hold at most 12!: 13! exceeds both signed and unsigned positive ranges, so guard n or use a wider result with checked multiplication.

Verification and further reading: [ARCH] multiply and branch/flag semantics; [ABI] register and stack discipline; [FURBER] chapters 3, 5 and 6.

11 Answered exam and classroom practice

Use the actual 3/9-mark structure. For a short answer, give a definition, the decisive mechanism and a concrete example or qualification. For a long ALP answer, give input/output contract, register plan, algorithm, commented code, result/trace and boundary cases. The suggested subdivisions below organize practice; they are not official marking schemes.

11.1 Forty short questions with model answers

S1. What is the role of Arm in an ARM-based chip?

Model answer. Arm supplies architecture and processor IP designs. A silicon vendor integrates a core with memory, buses and peripherals to make a chip; some architecture licensees implement their own compatible cores. The MCU datasheet, not the core name alone, defines peripheral addresses and pins.

S2. Which architecture does ARM7TDMI implement?

Model answer. ARMv4T: classic ARM-state instructions plus original Thumb. The number in ARM7TDMI is a core-family name, not architecture version 7. Cortex-M3 instead implements ARMv7-M.

S3. What is a load/store architecture?

Model answer. ALU instructions compute on registers/immediates; data memory transfers are explicit load/store operations. Use LDR, ADD, STR to add to a RAM variable. LDM/STM and other transfer forms also access data memory; ordinary ADD has no memory operand.

S4. How many registers are visible?

Model answer. Sixteen 32-bit core registers R0–R15, plus CPSR. Exception modes also expose their own SPSR. There are 31 physical core registers and six status registers, giving 37 total with banked copies.

S5. What does BL do to LR?

Model answer. It branches and saves the next instruction address in R14/LR. In ARM state at address A, LR=A+4. BX LR is an interworking-aware subroutine return; a nested call must not destroy the caller's saved LR.

S6. Predict flags after CMP with both operands 7.

Model answer. The discarded subtraction result is zero; N=0, Z=1, C=1 (no borrow), V=0. Thus EQ and HS are true, but HI is false because the operands are equal.

S7. What does C=0 after SUBS mean?

Model answer. An unsigned borrow occurred: the first unsigned operand was below the second. It does not alone prove signed negativity or signed overflow. Signed conditions also use N/V.

S8. How does CMP differ from SUB/SUBS?

Model answer. CMP subtracts, always writes arithmetic flags and discards the result. SUB stores the difference and keeps flags unchanged; SUBS stores it and updates flags.

S9. What does the S suffix mean?

Model answer. Request flag updates according to the instruction class. It does not mean signed arithmetic. Arithmetic writes NZCV; logical instructions obtain C from the shifter and preserve V; ARMv4T MULS has unpredictable C.

S10. List the seven modes.

Model answer. User, FIQ, IRQ, Supervisor, Abort, Undefined and System. All except User are privileged. Only the five exception modes have SPSRs; System shares the User bank.

S11. Why does FIQ bank R8–R12?

Model answer. Its private working registers reduce shared-register save/restore work. It also has banked SP/LR, higher priority than IRQ and the last vector slot. It still must preserve any shared registers it changes.

S12. Where is old CPSR saved on an exception?

Model answer. In the SPSR of the exception mode, such as SPSR_irq. ARM7 does not automatically stack all core registers. Reset is a special startup event, not an ordinary save-and-return exception.

S13. What is at vector address 0x18?

Model answer. The IRQ vector in the classic ARM7 low-vector table. It holds an ARM instruction that transfers control to a handler, rather than a raw Cortex-M-style handler pointer. FIQ is at 0x1C; 0x14 is reserved.

S14. Why does an ARM7 division ALP use a loop?

Model answer. ARMv4T has no hardware divide instruction. Repeated subtraction or binary long division builds it from existing instructions. A nonzero divisor, unsigned comparison convention and valid remainder contract must be stated.

S15. Divide unsigned R1 by 16 in one instruction.

Model answer. `MOV r0,r1,LSR #4`. It gives the floor of the unsigned quotient; mask R1 with 0x0F for the remainder. ASR on signed negatives has different rounding.

S16. Why is MUL with an immediate invalid?

Model answer. Multiply encoding takes register operands. Load the constant into a register first. On ARM7 choose a destination different from the encoded Rm and avoid PC. MUL keeps the low 32 bits.

S17. R0=CA; what does BIC with mask 0A produce?

Model answer. `CA AND NOT 0A = C0`, binary 11000000. Bits 1 and 3 are cleared; other bits are preserved.

S18. Set bit 5 without changing other bits.

Model answer. `ORR r0,r0,#0x20`. BIC with that mask clears bit 5, EOR toggles it and AND extracts it while clearing other positions.

S19. Trace SUB R0,R1,R2 for R1=10, R2=4.

Model answer. R0 becomes 6. R1 remains 10 and R2 remains 4 because they are distinct source registers. If a source is also the destination, that register is overwritten. Plain SUB does not update flags.

S20. Interpret MOV R0,R1,LSL #3.

Model answer. R0 receives R1 multiplied by 8 modulo 2^{32}. R1 stays unchanged if distinct from R0. Bits shifted beyond bit 31 are discarded, so overflow must be considered.

S21. Why does factorial's product start at 1?

Model answer. One is the multiplicative identity and produces the correct empty product 0!=1. Starting at zero forces every subsequent product to zero. A sum accumulator instead starts at zero.

S22. What if a loop's counter is never updated?

Model answer. If its initial comparison permits entry, it keeps testing the same value and may never terminate. Initialization, test and update must all be present. A watchdog resets it only if a watchdog is actually configured and allowed to expire.

S23. What is the largest one-word factorial?

Model answer. $12! = 479,001,600$ fits a positive signed or unsigned 32-bit word. $13! = 6,227,020,800$ does not; its low word is 1,932,053,504. Guard n or use a wider checked result.

S24. Why can most classic ARM instructions be conditional?

Model answer. ARMv4T ARM-state encodings have a 4-bit condition field tested against CPSR flags. Short sequences can avoid branches. Original Thumb does not have the same per-instruction condition field or the later IT instruction.

S25. What does a normal ARM-state PC read give?

Model answer. The executing instruction address plus 8 for ordinary uses. At 0x8000, MOV from PC reads 0x8008. BL's saved return address is still the next instruction, 0x8004. Avoid special PC-operand cases in simple ALPs.

S26. Which mode/state follows reset?

Model answer. Supervisor mode, ARM state, with IRQ/FIQ masked. Startup must initialize required stacks and hardware before unmasking. Do not assume all working registers or flags were initialized by reset.

S27. Assign an urgent source and ordinary sources to IRQ/FIQ.

Model answer. A supported controller can route the urgent source to FIQ and ordinary sources to IRQ. FIQ's extra register bank and last vector help latency. Verify the actual controller routing and masking rules; the core provides two request classes.

S28. Why is classic ARM attractive for embedded designs?

Model answer. A regular load/store ISA, efficient register datapath, pipeline and compact core can provide useful performance per area/energy. Thumb helps code density. Actual performance/power depends on the core, memory, clock and workload, not the word RISC alone.

S29. Compare System and User modes.

Model answer. They share R0–R15 and neither has an SPSR. System is privileged; User cannot alter privileged CPSR fields. System is not entered by its own exception.

S30. Why does a banked SP not guarantee a valid stack?

Model answer. It selects a private register copy but does not initialize its value or allocate writable RAM. Startup must assign each needed stack pointer. Shared working registers are still preserved by software.

S31. Distinguish ARM7 modes from Cortex-M modes.

Model answer. ARM7 has seven modes and banked exception registers/SPSRs, and supports ARM/original Thumb. Cortex-M uses Thread/Handler modes, MSP/PSP and hardware exception stacking; it executes a Thumb-family instruction set, not A32.

S32. Why is N alone not sufficient for signed less-than?

Model answer. Signed subtraction may overflow, flipping the apparent result sign. The signed LT test is N different from V; GE is N equal to V. Unsigned comparisons instead use C/Z.

S33. How does SBC propagate borrow?

Model answer. It subtracts the second operand and $(1 - C)$. After a low-word SUBS with a borrow, C=0, so the high-word SBC subtracts one extra. Do not subtract C itself.

S34. Is HALT a real ARM7 instruction?

Model answer. No. It is a classroom engine command. Complete examples use a terminal B loop; a subroutine returns through BX LR. Source END is an assembler directive in some tools, not a CPU halt.

S35. Why might MOV with a large constant fail?

Model answer. Ordinary ARM-state immediate encoding is an 8-bit value rotated by an even amount. Use MOV/MVN when encodable, or LDR=constant with a literal pool. ARM7 does not support MOVW/MOVT.

S36. Does ASR implement signed division exactly?

Model answer. It divides by a power of two rounding toward negative infinity. For -7 shifted by 1 it gives -4, whereas truncation toward zero gives -3. State the required convention and compensate or use a signed routine.

S37. Does ARM7TDMI have cache, MMU or NVIC?

Model answer. The bare core has no cache/MMU and no Cortex-M NVIC. The system/MCU can add surrounding memory and interrupt-controller blocks. Do not transfer Cortex or cached ARM-family features indiscriminately.

S38. Can every instruction be assumed one cycle?

Model answer. No. An ideal filled pipeline supports one simple ALU completion per cycle; memory transfers, branches, multiply, register shifts and wait states differ. $n + 2$ applies only to the stated ideal sequence.

S39. Why use SUBS PC,LR,#4 for an IRQ return?

Model answer. The IRQ LR is next instruction address plus 4. Subtracting 4 resumes the next instruction; writing PC with S in an exception mode restores CPSR from SPSR. A plain MOV/BX does not perform that complete return.

S40. Can User mode necessarily not access GPIO?

Model answer. Not from the core's mode rule alone. User cannot alter privileged CPSR fields, but access protection for peripheral memory depends on system hardware. Bare ARM7TDMI has no internal MPU/MMU enforcing a general peripheral sandbox.

11.2 Ten developed 9-mark practice answers

L1. Draw and explain ARM7 architecture, pipeline and a traced ADD

Prompt (3+3+3). Explain the datapath; draw the three-stage pipeline; trace ADD and the PC value.

Developed answer. Reproduce the vector datapath in Section 2: register reads feed operand A and the shifter's B path, the ALU computes, and a result bus writes the selected destination. Decoder/control chooses operations and checks conditions; address/memory logic fetches instructions and transfers data; the multiplier handles products. CPSR stores status. Show a unified instruction/data interface and avoid adding a cache/MMU to the bare core.

Draw F/D/E diagonals for three successive instructions. Explain latency versus throughput and explicitly assume single-cycle operations/no stalls for one completion per filled-pipeline cycle. An ideal ten-instruction sequence takes twelve cycles. For ADD R0,R1,R2 with 7 and 3, read operands, pass operand 2 unshifted, compute 10, write R0 and leave distinct sources/flags alone. At ARM address 0x8000 an ordinary PC read gives 0x8008. A taken branch flushes the sequential stream.

L2. Explain register banking, modes and IRQ entry/return

Prompt (3+3+3). List modes; explain banking; trace an IRQ.

Developed answer. Give the seven-mode table with correct mode bits and privileges. User/System share a bank and have no SPSR. FIQ banks R8–R14; IRQ/SVC/Abort/Undefined bank SP/LR. Count 31 physical core registers plus CPSR/five SPSRs=37. Banking selects a copy; stack initialization and shared-register preservation remain software duties.

For User CPSR 60000010 and next ARM address 8004, IRQ entry sets SPSR_irq=60000010, LR_irq=8008, CPSR=60000092 and PC=18. Hardware enters ARM/IRQ and masks IRQ while retaining F. Show software saving modified shared registers and servicing/acknowledging the controller source. SUBS PC,LR,#4 returns to 8004 and restores CPSR. If nesting is enabled, protect the existing LR/SPSR first. A simple mode write through MSR performs neither the save nor the vector jump.

L3. Explain flags, signedness and conditional execution

Prompt (3+3+3). Define NZCV; calculate two cases; find a maximum without branching.

Developed answer. N is bit 31, Z tests zero, C is unsigned carry/no-borrow, V is signed overflow. FFFFFFFF+1 gives result zero and NZCV=0110; 7FFFFFFF+1 gives 80000000 and NZCV=1001. State exact unsigned/signed interpretations rather than assigning signedness to a register. After CMP, signed GE tests N=V, LT tests N≠V; unsigned HS/LO test C.

Use CMP R1,R2; MOVGE R0,R1; MOVLT R0,R2 for signed maximum, or MOVHS/MOVLO for unsigned. The conditional MOVs omit S so the second tests the original comparison. Show FFFFFFFF versus 1: signed maximum is 1 while unsigned maximum is FFFFFFFF. Plain SUB followed by a conditional instruction is unreliable unless earlier flags are intentionally used.

L4. Compare RISC/CISC and explain advanced ARM features/versions

Prompt (3+3+3). Compare styles; explain three features; distinguish three architecture/core pairs.

Developed answer. Give fixed ARM-state encoding versus variable x86 encoding, register computation versus memory operands, and regular decode versus richer instruction encodings. Demonstrate a RAM update with LDR/ADD/STR and explain why instruction count alone cannot determine speed or power. Identify conditional execution, the barrel shifter and Thumb, with examples ADD R0,R1,R2,LSL #2 and BX-controlled state switching. Pipelining, banking and block transfers are further features.

Distinguish ARM7TDMI/ARMv4T from Cortex-M3/ARMv7-M and Cortex-A53/ARMv8-A. Cortex-M is Thumb-family with Thread/Handler; AArch64 has a different 64-bit register model. v5TE does not itself add Jazelle (J is required), and Thumb-2 is not present on all v6 cores. Keep the seven-mode/no-divider details attached to ARM7TDMI rather than all ARM products.

L5. Develop addition and subtraction ALPs with carry chaining

Prompt (3+3+3). Write simple addition/subtraction; extend one to 64 bits; explain a boundary case.

Developed answer. Use the complete `add.s` and `subtract.s` listings in Section 7, with R1/R2 inputs and R0 result. ADDS 25,17 yields 42/NZCV=0000; SUBS 30,12 yields 18/NZCV=0010. For a two-word sum use ADDS on low words and ADCS on high words. Trace 00000001:FFFFFFFF plus 00000002:00000001 to 00000004:00000000. For subtraction propagate $(1 - C)$ through SBC. Explain FFFFFFFF+1 versus 7FFFFFFF+1 to distinguish carry from signed overflow. A complete exam answer includes comments and the output register, not just mnemonics.

L6. Develop multiply ALPs and discuss truncation

Prompt (3+3+3). Use MUL; multiply by repeated addition; handle a wider product.

Developed answer. Give `multiply.s`: R1=12, R2=9, MUL R0,R1,R2, result 108. State register-only operands, conservative ARM7 destination restriction and low-word output; plain MUL preserves flags. For repeated addition initialize R0=0 and count R2 down with a zero test before adding R1, producing 84 for 12 times 7. Explain why zero is safe and a negative count violates the unsigned loop contract.

Use UMULL R0,R3,R1,R2 for 100000 squared. The full result is 00000002:540BE400, so a low-word-only MUL loses information. Nonzero high word indicates unsigned one-word overflow. MULS V is not a product-overflow flag. Signed full products need SMULL and a sign-extension check. Constant multiplication by 10 can use shifted ADD as shown in the chapter.

L7. Develop unsigned division and prove its remainder contract

Prompt (3+3+3). State architecture; write repeated subtraction; explain a full-width alternative.

Developed answer. ARM7TDMI has no divider. Give the complete guarded `divide_subtract.s` with Q in R0, remainder R1, divisor R2 and status R5. Test divisor zero, then CMP remainder,divisor and BLO done; SUB and increment Q until remainder is below divisor. For 47/6 the trace ends Q=7,R=5 and satisfies $47 = 7(6) + 5$. Explain why equality must not exit and why BLT is not a general unsigned test.

Repeated subtraction takes quotient-dependent iterations. Binary long division shifts in one dividend bit per pass and reduces the trial remainder at most once, maintaining $0 \leq r < B$. A

full 32-bit implementation needs the transient 33rd remainder bit, preserved by ADCS carry in `divide32.s`. It has 32 iterations but variable branch timing. Dividing unsigned by 2^k is LSR; signed ASR's rounding differs.

L8. Develop a factorial ALP, trace it and handle errors

Prompt (3+3+3). Design the registers/code; trace n=6; discuss zero and overflow.

Developed answer. Reproduce `factorial.s` in Section 10. Initialize product R0=1 and counter R2=1; retain n in R1, use R3 as legal multiply destination and R4 as status. Check unsigned n≤12, then multiply while i≤n and increment i. The top-of-loop invariant is product=$(i-1)!$. For n=6 the products are 1,2,6,24,120,720; i=7 exits.

n=0 skips the body and returns 1. n=13 is rejected because 13!=6,227,020,800 exceeds a word; an unchecked low word is 1,932,053,504. A zero product initializer and omitted counter update are separate defects. Changing the strict-greater exit to greater-or-equal skips multiplication by n. A callable extension must preserve LR/callee-saved registers and return a documented result/status.

L9. Design logical masks and trace a bit-count routine

Prompt (3+3+3). Explain four masks; trace the AA experiment; explain shifter carry.

Developed answer. AND extracts, ORR sets, EOR toggles and BIC clears selected bits. For AA, set bit 0 to AB, clear bit 7 to 2B, toggle bit 4 to 3B and extract the low nibble to 0B. Explain why AND with mask 80 keeps only bit 7 rather than clearing it.

Give the bitcount loop: MOVS R1,R1,LSR #1; ADC R0,R0,#0; SUBS count,count,#1; BNE loop, repeated 32 times. Carry from the shifted-out bit is consumed before count subtraction overwrites flags. CA has four set bits; FFFFFFFF has 32. For peripheral use, LDR/STR surround the logical computation and register semantics/concurrency must permit read–modify–write.

L10. Explain ISA encoding, memory and a subroutine call

Prompt (3+3+3). Decode an ARM word; explain literal loads/endianness; describe BL and return.

Developed answer. Draw the ordinary data-processing fields in Section 2.4. For ADD R0,R1,R2, cond=E, opcode=4, I=0, S=0, Rn=1, Rd=0, unshifted Rm=2, giving E0810002. Other instruction classes have different layouts. The word occupies little-endian bytes 02 00 81 E0, not reversed register bit positions.

An immediate is an 8-bit constant rotated by an even amount; 12345678 needs an assembler constant-loading strategy such as LDR=constant/literal pool. Branch over embedded pools. BL stores the next ARM instruction address in LR, while its displacement uses PC=A+8. Save LR before a nested BL and preserve callee-saved registers with a balanced aligned frame. BX LR is a normal return; an IRQ needs an SPSR-restoring exception return.

11.3 Six classroom laboratories: complete record mapping

The chapter programs provide full sources, register plans, algorithms and expected results. Use these record notes for the supplied debug challenges, extensions and viva questions.

Classroom lab	Complete program / expected result	Challenge and extension answer
1 Addition	`add.s`, Section 7.1; 25+17=42, NZCV=0000.	FFFFFFFF+1 gives zero, Z/C set. Three-number addition adds another operand; two-word addition uses ADDS/ADC.
2 Subtraction	`subtract.s`, Section 7.2; 30-12=18, NZCV=0010.	Reversed inputs give FFFFFFEE, N=1/C=0. Two-word subtraction uses SUBS then SBC.
3 Multiplication	`multiply.s`, Section 7.4; 12 times 9=108.	100000 squared needs high word 2; use UMULL. Multiply by ten with shifts/adds.
4 Division	`divide_subtract.s`, Section 9; 47/6 gives 7 remainder 5.	Divisor zero returns status=1. Equality must still subtract; full-width `divide32.s` supplies the extension.
5 Logical operations	`logic.s`, Section 8; R0=3B, R1=0B.	Wrong AND 80 gives R0=90 after later EOR, R1=00. Swap a byte's nibbles with shifts and masking.
6 Factorial	`factorial.s`, Section 10; 6!=720.	Reject n=13, preserve 0!=1; callable extension is `factorial_call.s`.

Viva answers, lab 1. S makes flag writes explicit; ADC includes old carry; addition C means unsigned carry out. **Lab 2.** Subtraction C means no borrow; CMP discards the difference; V means signed range overflow. **Lab 3.** MUL has no immediate field; multiply by eight with LSL 3; UMULL/SMULL preserve unsigned/signed 64-bit products. **Lab 4.** The selected core omits divide hardware; later hardware capability is architecture-specific; unsigned divide by sixteen uses LSR 4. **Lab 5.** AND keeps mask-selected bits, BIC clears them; TST mask 20 tests bit 5 without changing the value; permitted read–modify–write preserves unrelated peripheral bits. **Lab 6.** Start at multiplicative identity 1; 12! is the limit; guard n or inspect a full product for overflow.

11.4 Six debugging exercises with corrected diagnoses

1. **Factorial returns zero.** `MOV r0,#0` initializes the product incorrectly. Replace with 1; 5! then gives 120.
2. **Sum 1–10 does not terminate.** Add `ADD r1,r1,#1` *before the existing branch*. Replacing that branch with the increment and falling through to done would sum only one iteration. The correct sum is 55.
3. **Division exit uses BLE.** For the shown 47/6 it still gives 7,5. For 48/6 it stops at 7,6 instead of 8,0, so the quotient is *too small*. Use BLO for unsigned operands; equality must continue. The supplied symptom/diagnosis is inconsistent.
4. **AND 40 intended to clear bit 6.** It keeps only that bit. For D6, use BIC 40 to obtain 96.
5. **Conditional MOVEQ follows plain SUB.** Flags are stale; use SUBS/CMP. With equal inputs, Z becomes one. The unchanged prior Z could otherwise make the bug appear intermittent.

6. **LSR 1 intended to double 21.** It gives 10. Replace with LSL 1, giving 42; remember that larger shifts can discard high bits.

11.5 All fifteen graded practice problems, completed

Beginner B1–B5. These preserve the classroom tasks while giving explicit outputs/contracts.

```
@ B1: load 25, copy: r5=r6=25
MOV r5, #25
MOV r6, r5
@ B2: r2 = old r1 + 100, modulo 2^32
ADD r2, r1, #100
@ B3: two ways to make r0 zero, without setting flags
MOV r0, #0
EOR r0, r0, r0
@ B4: result r2 is explicitly 0 or 1
MOV r2, #0
CMP r1, #10
MOVEQ r2, #1
@ B5: multiply by 4 modulo 2^32
MOV r0, r1, LSL #2
```

Intermediate I1–I5.

```
@ I1: (a+b)*2, a=r1, b=r2
ADD r0, r1, r2
MOV r0, r0, LSL #1
@ I2: set bits 0 and 3, clear bit 7
ORR r0, r0, #0x09
BIC r0, r0, #0x80
@ I3: subtract 1 from zero
MOV r1, #0
MOV r2, #1
SUBS r0, r1, r2          @ FFFFFFFF; NZCV=1000
@ I4: countdown from 10 to 0
MOV r1, #10
count_loop:
SUBS r1, r1, #1
BNE count_loop
```

I4 requires a positive initial count or a precheck; starting at zero would wrap and execute billions of iterations. **I5**, sum 1 to n for $0 \leq n \leq 65535$, uses the following counted loop; the bound keeps the largest sum within signed 32-bit range. It leaves zero for n=0.

```
MOV r0, #0
MOV r2, #1
sum_loop:
CMP r2, r1
BHI sum_done
ADD r0, r0, r2
ADD r2, r2, #1
B sum_loop
sum_done: B sum_done
```

For n=10 the sum is 55; for n=65535 it is $65535(65536)/2 = 2,147,450,880$.

Challenge C1–C5. C1 is signed maximum: CMP, MOVGE, MOVLT (unsigned variant MOVHS/MOVLO), explained in the flags chapter. C2 is the complete `bitcount.s`; its carry rule corrects the classroom engine. C3 is the complete full-width `divide32.s`, with both quotient and remainder. C4 is the completed `reverse4.s`, including mask and all four iterations, replacing the HTML's partial solution. C5 is `gcd.s`, using unsigned conditions and zero guards; the classroom positive-small-input version uses signed conditions and does not handle zero safely. Every challenge now has complete logic, a result convention and a testable source.

11.6 Classroom quiz and demonstrations: where each answer lives

The 28 JavaScript quiz items are answered in order by **S1–S28** above, with qualifications/corrections where required. The clickable register explorer becomes the register-role/banking tables; flag presets become the worked NZCV table; encoding choices become the three independently checked machine words. The core animation becomes the datapath/ADD trace; the IRQ/FIQ/SWI/undefined animation becomes the exception table and worked IRQ entry. Division and factorial runners become complete programs and iteration tables; the seven-instruction execution visualizer becomes the source/destination/flag explanations.

Application demonstration	Static learning connection
Washing machine	MCU hierarchy, sensor reads, mode-based interrupt service and driver-mediated outputs.
ABS controller	Deadline-sensitive control, interrupt class choice and arithmetic/conditional decisions.
Fitness band	Low-energy design, interrupt-driven work, bit packing and sensor averaging.
Robot	Encoder/timer events, position arithmetic and bounded motor commands.
IoT endpoint	UART status tests, logical masks, protocol decisions and appropriate fault handling.

These connect architecture to applications; they are not new peripheral-driver chapters. A watchdog event is device-specific and does not necessarily enter an ARM Abort/Undefined exception: it commonly resets a system or raises a configured interrupt. A hardware fault only causes an abort if the system actually signals it to the core.

12 Final revision sheet and completion checklist

Architecture and the core

ARMv4T is an architecture; ARM7TDMI is its implementation; LPC2148 is an MCU using ARM7TDMI-S. TDMI: Thumb, Debug, Multiplier, EmbeddedICE. ARM7 is not ARMv7. Core: register bank, ALU, shifter, multiplier, decoder/control, memory/address logic. ARM7TDMI has a unified interface and no built-in cache/MMU/NVIC/divider.
ARM-state instructions are 32 bits; original Thumb is mostly 16-bit (BL is a two-halfword sequence). Fetch/decode/execute overlap; $n + 2$ cycles is an ideal no-stall single-cycle-execute model. Normal ARM PC read=$A + 8$; BL LR=$A + 4$. Taken branches refill the pipeline.

Registers, flags and conditions

R0–R12 general; R13=SP, R14=LR, R15=PC. 31 physical core registers + CPSR + five SPSRs=37. User/System share a bank and have no SPSR; FIQ banks R8–R14; IRQ/SVC/Abort/Undefined bank SP/LR. Initialize every stack you use.
CPSR: N31 Z30 C29 V28, I7/F6 are interrupt-disable bits, T5 selects ARM/Thumb, mode4:0. ADD/SUB without S preserve flags; CMP always sets them. C after subtraction means **no borrow**. Logical S uses shifter C and preserves V; ARMv4T MULS C is unpredictable.
EQ Z; NE !Z; HS C; LO !C; HI C $\wedge$!Z; LS !C $\vee$ Z. Signed GE $N = V$, LT $N \neq V$, GT !Z and $N = V$, LE Z or $N \neq V$. S is not "signed".

Modes, vectors and returns

User10, FIQ11, IRQ12, SVC13, Abort17, Undefined1B, System1F (hex). Only User is unprivileged. Reset: SVC, ARM, IRQ/FIQ masked. Exception: old CPSR$\rightarrow$SPSR, return address$\rightarrow$banked LR, select mode, T=0, mask IRQ, branch to vector. Shared registers are saved by software.
Vectors: 00 reset, 04 undefined, 08 SWI, 0C prefetch abort, 10 data abort, 14 reserved, 18 IRQ, 1C FIQ. IRQ/FIQ return SUBS PC,LR,#4; SWI/undefined MOVS PC,LR; retry data abort SUBS PC,LR,#8. MSR mode change does not save exception state. Use BX for normal interworking return, not as a substitute for restoring SPSR.

ALPs and the common traps

ADD/ADC chain carry; SUB/SBC chain $(1 - C)$. MUL has register operands, low-word output and conservative ARM7 Rd$\neq$Rm; UMULL/SMULL give full products. ORR sets, BIC clears, EOR toggles, AND masks. MOVS shift writes bit carry; consume it before another flag writer.
Ordinary immediate = 8 bits rotated right by an even amount. Use LDR=constant/literal pool for large values; no MOVW/UDIV on ARM7. LSL multiplies modulo 2^{32}, LSR divides unsigned by powers of two, ASR rounds signed negatives down. Register-count shift rules differ at 32 and above.
Unsigned divide: test divisor zero; continue subtracting on equality; use unsigned branches. Full-width long division needs transient carry, and 32 iterations do not imply fixed cycles. Factorial: product=1, include counter n, 0!=1; guard n$\leq$12. 12!=479,001,600; 13! overflows. Real program stops at B done; HALT is a classroom command.

12.1 Syllabus completion and self-test checklist

Mark each item only when you can explain it without viewing the answer. The labels point to the full teaching sections.

	Required skill	Section
☐	Distinguish architecture, core, MCU and embedded system.	1
☐	Explain the ARM core and a real MCU's surrounding blocks.	1.2, 1.3
☐	Compare RISC/CISC using a memory-update example.	1.4
☐	Explain conditional execution, shifting, Thumb and other advanced features.	1.5
☐	Map core names to versions/profiles without mixing ARM7 and ARMv7.	1.6
☐	Draw the datapath and three-stage pipeline with valid timing assumptions.	2
☐	Explain ISA fields and encode a simple data-processing word.	2.4
☐	Name registers; explain GP roles, SP/LR/PC and banking/counts.	3, 3.1
☐	Decode CPSR and distinguish state from operating mode.	3, 5
☐	List seven modes, privilege, banks and SPSRs accurately.	5
☐	Trace exception entry/return and explicit mode switching.	5.3
☐	Compute NZCV and choose signed/unsigned conditions.	4
☐	Write and explain addition/subtraction, including carry/borrow chaining.	7.1, 7.2
☐	Write MUL/repeated-addition ALPs and explain full versus low products.	7.4
☐	Set/clear/toggle/test bits and trace logical programs.	8
☐	Write guarded division and verify $A = QB + R, 0 \leq R < B$.	9
☐	Write factorial, trace every pass, handle 0 and reject overflow.	10
☐	Diagnose classroom defects and answer each lab/viva/practice item.	11

Final closed-book rehearsal. Draw CPSR and the bank table; derive a borrow and an overflow example; write the IRQ return and explain both effects; write addition, subtraction, multiplication, logical masks, guarded division and bounded factorial from a register plan. Check results for zero, equality and a boundary value. Read the discrepancy appendix before using the classroom engine as an answer checker.

13 Discrepancies and deliberate teaching simplifications

The classroom HTML is useful teaching input, but its prose, generated data and simulator are not always consistent. Corrections below were checked against primary manuals and, where appropriate, direct execution of the supplied JavaScript. The existing local 19-page summary was treated similarly. None of these changes edits the original teaching files.

Classroom statement/behaviour	Verified treatment	Type / support
Every privileged mode has banked SP/LR and SPSR.	System is privileged but shares User registers and has **no SPSR**. The generated MODES explorer gets this right; slide 9 prose/revision generalization does not.	Actual prose error. [TRM], [ARCH].
Handler starts with a valid stack automatically.	A bank is selected automatically; startup must previously initialize that SP to valid RAM. No automatic shared-register stacking on ARM7.	Omitted prerequisite. [TRM].
Modes structurally prevent all peripheral reconfiguration by User.	CPSR control writes are privileged. Peripheral memory access needs system protection; bare ARM7TDMI has no built-in MMU/MPU.	Too broad. [TRM], [ARCH].
Only LDR and STR ever access memory.	ALU computation is register-based; additional data-transfer forms include LDM/STM, byte/halfword forms and SWP. Instruction fetch also reads memory.	Teaching shorthand. [ARCH].
REP LODSD and an ARM sum loop perform the same task.	REP LODSD repeatedly loads EAX; it does not accumulate the loaded values into a sum. Use a real summation algorithm when comparing.	Actual example mismatch; source operation explicitly loads.
Fixed encoding guarantees cycles, no fetch stalls, or speed/power superiority.	Branches, multiply, register shifts, memory wait states and implementation affect timing. Encoding regularity is a design advantage, not a universal result.	Overstatement. [TRM] cycle tables.
Every barrel shift is free.	Immediate ALU operand shifts normally add no extra shift cycle; register-controlled shifts add an internal cycle on ARM7.	Missing qualifier. [TRM].
v5TE adds Jazelle; v6 generically has Thumb-2.	Jazelle needs J (e.g. v5TEJ); Thumb-2 is a v6T2 feature, not every v6 core. M-profile versions are separate.	Actual timeline conflation. [PROFILE], [ARCH].

Classroom statement/behaviour	Verified treatment	Type / support
The same ISA program runs on any modern ARM.	Match ISA subset, state and software environment. A32 is not executable on Thumb-only Cortex-M or directly in AArch64.	Compatibility overstatement. [PROFILE].
Original Thumb uses IT for conditional execution.	ARMv4T original Thumb lacks IT. It has conditional branches; IT is from later Thumb-2-era architectures.	Architecture mismatch. [ARCH], [PROFILE].
Simulator HALT, arbitrary immediates, sequential PC, no memory or timing.	These are explicitly documented educational choices. Real files use B done, ARM7 immediate rules, real linked addresses and aligned memory; timing claims remain qualified.	Intentional simplifications, not hidden errors.
Shift/logical S updates only N/Z in the JavaScript engine.	Real logical S also writes shifter carry. The supplied popcount uses that carry but the engine never supplies it: shifting 1 right and ADC yields 0 there, versus 1 on hardware semantics.	Actual simulator gap. [ARCH]; direct engine audit.
JavaScript LSL count 32 behaves like count 0.	Register-controlled ARM LSL by 32 yields zero (carry old bit 0); JavaScript shift masking yields the unchanged value. Shift edge cases need architectural rules.	Actual simulator gap. [ARCH]; engine audit.
Simulator accepts ARM7 MUL destination/source overlap.	Examples retain the classic conservative Rd≠Rm rule. DDI 0100I notes implementation relaxations; GNU as may swap source operands. Source acceptance alone does not establish the actual encoding or target behaviour.	Legacy rule/tool nuance. [ARCH]; disassembly checks.
Division lab uses BLT as an unsigned test.	It works on the stated small positive operands, but full-range unsigned division needs BLO/BCC. Both routines now guard zero.	Small-input simplification with wider-input defect. [ARCH].
Bug 3: BLE makes 47/6 quotient too large.	Both BLE and BLT give 7,5 for 47/6. BLE fails on equality: 48/6 gives 7,6 (quotient *too small*), versus 8,0.	Actual symptom/diagnosis error; direct engine test.
Factorial BLT instead of BGT yields (n-1)!.	In the shown top-tested loop BLT exits immediately for ordinary n>1. BGE instead of BGT skips the last factor; in unsigned form BHS instead of BHI does so.	Actual branch diagnosis error; loop trace.

Classroom state-ment/behaviour	Verified treatment	Type / support
One watchdog eventually resets every infinite loop.	Only an enabled, correctly configured watchdog allowed to expire does that. Watchdog routing may reset or interrupt; it is not automatically an Abort exception.	Omitted device condition. [NXP].
All six-step exception entries end by returning PC directly from LR.	Return offset depends on exception; restore CPSR from SPSR as well. Reset is special. IRQ/FIQ use LR-4, data-abort retry LR-8, SWI normally LR.	Animation simplification. [TRM].
FIQ never needs to save anything.	Private R8–R12 reduce saves, but shared registers used by the handler still need preservation. Same-mode nesting also needs saved LR/SPSR.	Missing handler contract. [TRM].
Four-bit reversal answer ends with an ellipsis; divide challenge refers elsewhere.	Complete four-bit reversal and full-width divide programs, boundary checks and results are supplied here.	Incomplete solutions filled, not claimed errors.
Source registers never change.	Distinct source-only registers stay unchanged; a source used as destination or a load's writeback base changes.	Missing qualifier. [ARCH].

Positive carry-over. The HTML correctly emphasizes ARMv4T/ARM7TDMI assumptions, no-borrow subtraction, flag-setting suffixes, factorial initialization, architecture-dependent division and the educational engine boundary. Those are retained. Its small examples remain useful once the stated limits and the corrections above are understood.

14 Chapter-mapped sources and further reading

Primary references were used for architecture/register/instruction claims. Links below are direct documents or topic-specific official manuals, checked on 7 October 2026. The downloadable Arm PDFs were read locally when the browser viewer could not fetch them. Document identifiers and chapter/section names are more reliable than guessed textbook page numbers.

14.1 Official scope and local teaching sources

Syllabus: the supplied *S3–S8 Electrical and Computer Engineering Final Syllabus*, physical pages 128–131, PCEOT501, Module 4 on 129; pattern on 130. Its row controls scope. **Classroom:** Prof. Murali Krishnan's `mces_module4.html`; all 23 slide sections, JavaScript arrays and engine inspected. **Supplement:** supplied `Module 4/Module-4.pdf`, 19 pages, reviewed but not treated as a primary architectural authority. Their original files remain unchanged. The portable source bundle includes teaching inventories and the audit results, not large third-party manuals.

14.2 Architecture, device and tool references

1. **[TRM] Arm, ARM7TDMI Technical Reference Manual, DDI 0210C.** Introduction/instruction summaries (chapter 1), programmer's model/registers/PSRs/exceptions (chapter 2), instruction-cycle timing (chapter 6). Supports foundations, datapath, modes, vector/return rules and timing qualifications.

   `https://documentation-service.arm.com/static/5f4786a179ff4c392c0ff819`

2. **[ARCH] Arm Architecture Reference Manual, DDI 0100I.** Read ARMv4T-applicable rules: A2 programmer's model, A3 condition field, A4 instruction definitions, A5 data-processing and transfer addressing. This manual also describes later architectures; version annotations must be observed. Supports flags, encodings, banking, MRS/MSR, operand restrictions and all ALPs.

   `https://documentation-service.arm.com/static/5f8dacc8f86e16515cdb865a`

3. **[NXP] NXP LPC2141/42/44/46/48 datasheet.** Features, memory, block diagram and interrupt/watchdog overview. Supports the concrete ARM MCU/system distinction. No device-specific peripheral driver is claimed in the core-only programs.

   `https://www.nxp.com/docs/en/data-sheet/LPC2141_42_44_46_48.pdf`

4. **[GNU] GNU Binutils, Using as: ARM directives and syntax.** CPU/state directives, labels, comments, encoding selection and pseudo-instructions. Supports the actual GNU sources and reproducible assembly commands.

   `https://sourceware.org/binutils/docs/as/ARM-Directives.html`

   `https://sourceware.org/binutils/docs/as/ARM-Syntax.html`

5. **[ASM] Arm Compiler v6.11 armasm User Guide, 100069_0611_00_en.** Source structure and directives (chapters 2 and 7), Unified Assembler Language, literal pools and stacks (chapter 3). Used for traditional armasm syntax comparisons; instruction behavior is checked against ARCH/TRM, not inferred from a newer compiler. No licensed armasm execution is claimed.

   `https://documentation-service.arm.com/static/5f75fe0b0ccfbc33efcab7d8`

6. **[ABI] Arm, Procedure Call Standard for the Arm Architecture (AAPCS32).** Core-register roles and stack discipline; architectural banking is independent of this software convention. Supports the callable factorial example.

   `https://github.com/ARM-software/abi-aa/blob/main/aapcs32/aapcs32.rst`

7. **[PROFILE] Arm, compiler processor/architecture compatibility table and Cortex-M resources.** Distinguishes ARM7TDMI/v4T, v5TE versus v5TEJ, v6 versus v6T2, and Cortex-M profiles. Later families are comparison context, not extensions silently assumed for the core under study.

   https://documentation-service.arm.com/static/5ea04d399931941038de55af

   https://developer.arm.com/community/arm-community-blogs/b/architectures-and-processors-blog/posts/cortex-m-resources

14.3 Prescribed textbooks: honest access and chapter mapping

8. **[FURBER] Steve Furber, ARM System-on-Chip Architecture, 2nd edition.** The syllabus names this book without an edition. The publisher's second-edition contents were checked: chapter 1 *An Introduction to Processor Design*; 2 *The ARM Architecture*; 3 *ARM Assembly Language Programming*; 4 *ARM Organization and Implementation*; 5 *The ARM Instruction Set*; 6 *Architectural Support for High-Level Languages*; 7 *The Thumb Instruction Set*; 9 *ARM Processor Cores*; 11 *Architectural Support for Operating Systems*; 13 *Embedded ARM Applications*. These are reading recommendations by chapter title, not claims that inaccessible chapter bodies were consulted.

   https://www.pearson.com/en-gb/subject-catalog/p/arm-system-on-chip-architecture-arm-system-on-chip-architecture/P200000004250/9780201675191

9. **[DAS] Lyla B. Das, Embedded Systems: An Integrated Approach.** Prescribed for general embedded-system context. Publisher catalogue information was available; an exact ARM chapter map/body was not available and is not invented. The other prescribed books by Mazidi, Ayala and Steiner chiefly support the preceding 8051 modules; they are not substituted for an ARMv4T architecture reference.

   https://in.pearson.com/content/dam/region-growth/india/pearson-india/Support/pdf/Catalog-ECE-2019-FINAL.pdf

14.4 What to open for each chapter

Guide chapter	Recommended source sections
ARM, core, MCU, RISC, features and versions	TRM introduction; NXP features/block diagram; PROFILE compatibility table; Furber 1, 2, 9, 13.
Datapath/pipeline/ISA encoding	TRM 1 and 6; ARCH ARM encodings/addressing; Furber 2, 4, 5.
Registers/CPSR/flags	TRM 2; ARCH A2/A3 and instruction flag effects; Furber 2 and 5.
Modes/exceptions/mode switching	TRM 2 exception tables; ARCH MRS/MSR and exception return; Furber 2, 5, 11.
Assembly, immediates, memory and calls	GNU ARM syntax/directives; ASM constants/literal pools/stacks; ABI register/stack rules; Furber 3, 5–7.
Arithmetic, logic, division and factorial	ARCH operation definitions/shift flags; TRM instruction/cycle tables; Furber 3 and 5. Algorithms/traces are independently authored and tested.
Practice, corrections and revision	The relevant primary sections above, the complete source listings and generated classroom/verification inventories.

Verification limits to keep in view. Functional emulator execution does not validate ARM7 bus cycles, exception latency, real IRQ/FIQ sources, electrical operation or a complete boot image. The structural IRQ fragment was assembled and manually checked against the exception table; it has no device service/acknowledge code and was not represented as a board-tested handler. No claim is made of native Keil ARM compilation or unavailable textbook-body access. The source README explains reproducible checks.